



2026 年陕西师范大学研究生数学建模竞赛

校内赛论文

培养单位 数学与统计学院

参赛队名 你说得都队

指导老师 张建国

队长姓名 闫心雨

队员姓名 1.李喜平

2.白姣姣

研究生院（党委研究生工作部）

数学与统计学院

2026 年陕西师范大学研究生数学建模竞赛校内赛论文

题目：基于多目标优化的老年福利慰问金分配与调整模型

摘要：

随着人口老龄化程度持续加深，老年福利慰问金科学公平分配与动态政策调整成为民生保障与社会治理的重要课题。针对 A 市月度预算约束、补助发放规范、困难群体帮扶、高层次人才激励以及行政信赖保护等多重现实要求，本文围绕三种既有分配方案评价、最优分配方案设计、追加预算约束调整、分配政策条款构建四大问题，开展量化建模与优化研究，旨在建立一套规则透明、分配公平、政策导向明确、可落地实施的老年慰问金分配与调整体系。

首先，针对问题一，本文对三种初始分配方案进行了对比评价。采用**多元线性回归模型与决策树模型**，从模型拟合优度、整体显著性、核心变量影响机制及分配规则逻辑四个维度，综合分析各方案的公平性与政策契合度，最终筛选出方案三作为最优初始分配方案，该方案分配逻辑清晰，能有效向低收入困难老人与高层次人才倾斜，与政策目标高度契合。

其次，针对问题二的精细化分配与预算整数约束难题，构建“**MPI 多维筛查+AHP-熵权融合权重评价+改进最大余额法**”的两阶段混合分配模型，在实现刚性兜底的同时，通过主客观融合权重计算分配优先级，并采用改进最大余额法，在满足百元整数倍约束下实现预算零误差闭环分配，并且通过不同 λ 值的对比分析验证了模型的鲁棒性。

在此基础上，针对问题三的追加预算与多重约束场景，沿用问题二的成熟模型，构建**带残差补偿的改进最大余额分配模型**，在满足人数限制、金额上限与信赖保护原则的前提下，实现追加预算的精准优化配置。

最后，针对问题四，本文提炼形成了规范化的**分配与动态调整政策条款**，并复用已有模型框架求解新政策下的分配方案，与原调整方案进行多维度对比验证。研究结果表明，本文构建的模型在不同预算与政策约束场景下均表现出良好的鲁棒性，能够同时实现弱势老人兜底帮扶与高层次人才政策激励的双重目标，且完全满足所有发放规则与行政约束要求。

本文的创新点主要体现在：一是构建了“**MPI 多维筛查+AHP-熵权融合权重+改进最大余额整数优化**”的闭环分配框架，兼顾了数据客观性与政策导向性；二是引入**残差补偿机制**，解决了百元整数约束与预算闭环的优化难题；三是将信赖保护原则量化为可计算的边界约束条件，实现了政策调整的平稳过渡。该模型可为城市老年福利慰问金的分配及政策优化提供科学的量化参考与决策依据。

关键词：老年福利慰问金 多维贫困指数 AHP-熵权融合权重 改进最大余额法 多目标优化

目录

一、问题重述.....	4
1.1 问题背景.....	4
1.2 问题要求.....	4
二、问题分析.....	4
2.1 问题一的分析.....	4
2.2 问题二的分析.....	5
2.3 问题三的分析.....	5
2.4 问题四的分析.....	5
三、模型假设.....	5
四、定义与符号说明.....	6
五、模型的建立与求解.....	6
5.1 数据预处理.....	6
5.1.1 数据清洗与整合.....	7
5.1.2 标准化处理.....	7
5.2 问题一模型的建立与求解.....	7
5.2.1 三种方案的合规性检验.....	7
5.2.2 多元线性回归分析.....	8
5.2.3 决策树规则挖掘.....	9
5.3.4 模型综合分析结论.....	11
5.3 问题二模型的建立与求解.....	11
5.3.1 多维贫困兜底识别 (MPI).....	12
5.3.2 加权评分分配模型.....	13
5.3.3 改进的最大余额法预算.....	16
5.3.4 模型最终结果与评价分析.....	17
5.3.5 模型稳健性与参数敏感性分析.....	18
5.4 问题三模型的建立与求解.....	20
5.4.1 调整对象筛选与信赖保护锁定.....	21
5.4.2 带边界拦截的首轮最大余额分配.....	21
5.4.3 残差二级边际优化与预算闭环.....	22
5.4.4 模型求解结果.....	22

5.5 问题四模型的建立与求解.....	22
5.5.1 规范化分配和调整政策条款	22
5.5.2 基于第二问优化模型的分配方案求解	23
5.5.4 分配方案对比分析.....	25
六、模型的评价与优化	26
6.1 模型优点	26
6.2 模型不足	26
6.3 模型改进与优化方向	26
参考文献.....	28
附录.....	28

一、问题重述

1.1 问题背景

随着我国人口老龄化进程的加速，如何科学、合理地分配老年福利慰问金，成为保障老年人权益、促进社会公平的重要议题。当前，慰问金分配不仅需要兼顾对有突出社会贡献老人的表彰、对生活困难老人的帮扶以及对孤寡老人的关怀，还需严格遵循行政法的信赖保护原则，确保政策的稳定性与连续性，避免因政策变更对老年人造成不必要的利益损失。因此，构建一套具备可解释性、公平性与稳定性的智能化分配与调整系统，具有重要的现实意义与应用价值。

在此背景下，A 市计划建立一套老年福利慰问金的智能化分配与调整系统，旨在通过系统化的算法设计，形成一套透明、公正且易于民众理解的老年福利政策。

1.2 问题要求

A 市每月可支出 20 万元用于老年慰问金与补助分配，同时设定了明确的发放规则：分配金额需为 100 元的整数倍；原则上每位普通老人每月分配金额不超过 2000 元（国家、省、市级人才不受此限制）；为保障覆盖面，至少一半参与分配的人员应获得不低于 100 元的补助。本年度 A 市参与分配的老年人员信息及三种初始分配方案数据见附件 1。

现需围绕以下四个核心问题展开研究：

问题一：结合附件 1 的老年人员信息数据，分析三种分配方案的特点，并从合规性、公平性、政策导向性等多维度评价方案的合理性，选出最优方案。

问题二：若由我们设计分配方案，需从社会贡献、低保扶贫、孤寡老人关怀等多个角度出发，构建多目标优化模型，设计一套兼顾公平与效率的最优分配方案。

问题三：目前 A 市已按照分配方案二实行了数月的分配，政府计划将接下来几个月的分配总金额提升至 30 万元/月。结合行政法的信赖保护原则，需满足以下约束：（1）对于已获得分配的人员，至多可调整其中 20% 人员的每月分配金额；（2）调整后，个人分配金额占总金额的前后比例变化不超过 50%。请在此基础上设计分配与调整策略，并形成具体的分配方案。

问题四：为了提升分配政策的公开透明度与民众接受度，需围绕最优分配方案、信赖保护原则等核心要求，建立几条简洁明了的分配与调整政策条款。在问题三的基础上，计算该政策下的分配方案，并与问题三的结果进行对比分析，验证政策的有效性与优越性。

二、问题分析

2.1 问题一的分析

问题一要求结合老年人员信息数据，分析三种已有分配方案的特点并评价其合理性。其核心是对现有分配方案进行多维度的诊断与对比。首先，需对三种方案进行合规性检验，验证其是否满足总预算不超过 20 万元、分配金额为 100 元整数倍、普通老人补助不超过 2000 元、受助老人比例不低于 50% 等硬性约束。其次，需从政策导向匹配度出发，分析各方案对“有社会贡献老人、低保困难老人、孤寡老人”三类重点群体的倾斜程度，判断其是否真正体现了政策初衷。此外，还需通过线性回归及决策树等方法，挖掘各方案的内在分配逻辑，评估其公平性与可解释性，最终综合判定最优方案。建模难点在于：如何从复杂老人特征

中挖掘三种分配方案背后隐藏的分配规则，避免仅凭主观经验评价。关键技术是利用多元线性回归识别显著影响因子，并结合决策树模型解析层级分配逻辑，从合规性、公平性、政策导向性实现对三种方案的定量对比与择优筛选。

2.2 问题二的分析

问题二要求从社会贡献、低保扶贫、孤寡关怀三个角度出发，设计最优分配方案。这是一个典型的多目标优化问题，其核心是在预算约束下，实现对三类重点群体的精准帮扶与资源的高效配置。首先，需将“社会贡献、低保水平、孤寡状况”等定性指标转化为可量化的评分体系，并确定各维度的合理权重。其次，构建了以预算上限、补助金额限制、受助比例等为约束条件的优化模型。通过求解该模型，得到一套兼顾公平性、导向性与合规性的最优分配方案，确保有限的资金能发挥最大的社会效益。建模难点在于：如何兼顾多群体、多指标、多目标的复杂需求，同时满足百元整数、预算严格封顶等硬性整数约束。关键技术是引入 MPI 多维贫困指数实现特困老人精准兜底，通过 AHP-熵权融合权重构建综合评分体系，并采用改进最大余额法解决比例分配无法适配整数约束与预算闭环的难题。

2.3 问题三的分析

问题三要求在方案二的基础上，将月预算提升至 30 万元，并结合行政法的信赖保护原则设计调整策略。其核心难点在于平衡政策稳定性与资金增量带来的优化空间。根据题目约束，仅可调整不超过 20% 已分配人员的金额，且调整前后个人补助占总金额的比例变化不超过 50%。因此，调整策略的关键在于：首先锁定 80% 老人的原有补助金额以保障政策稳定性；其次，将新增的 10 万元资金与剩余 20% 的抚恤金补贴优先用于提升低保、孤寡及高贡献老人的补助；最后，在满足比例变化约束的前提下，对部分老人的补助进行微调，形成既符合信赖保护原则，又实现资金增量效益的分配方案。建模难点在于：在预算扩容的同时，严格平衡信赖保护约束、人数调整上限与金额波动限制，避免政策变动幅度过大。关键技术是将信赖保护原则量化为比例约束区间，采用锁定保护、边界拦截与残差微调的改良最大余额策略，在不突破约束红线前提下实现追加预算的平稳优化配置。

2.4 问题四的分析

问题四要求围绕最优分配方案与信赖保护原则，建立公开透明的分配政策，并与问题三的结果进行对比分析。其核心是将优化模型转化为民众易于理解的政策条款，并验证其有效性。首先，需提炼出简洁明了的分配规则，明确补助优先级、金额梯度及调整原则。其次，依据该政策条款重新计算分配方案，并从合规性、政策导向性、可解释性等维度，与问题三的调整方案进行对比，验证新政策在提升透明度与公平性方面的优势。建模难点在于：如何把复杂数学模型转化为民众易懂、可落地的政策条款，并科学量化两套分配方案的优劣。关键技术是构建多维度评价指标体系，从合规性、公平性、政策导向、稳定性等角度建立对比评价标准，定量验证新政策方案的有效性与优越性。

三、模型假设

为了对老年福利慰问金分配与调整问题进行合理简化，我们建立了以下模型假设：

- 1、假设附件 1 中的老年人员信息数据真实可靠，能够准确反映老人的社会贡献、生活水平、孤寡状况等特征，无数据缺失或错误。
- 2、假设政策实施期间，所有老人的身份状态（如是否为低保户、孤寡老人、

高层次人才) 保持不变, 不发生变动。

3、假设行政法信赖保护原则的量化约束(调整人数不超过 20%、比例变化不超过 50%) 仅针对已获得分配的老人, 未获得补助的老人不受此限制^[1]。

4、假设慰问金按月一次性发放, 不考虑跨月结转、延迟发放等情况, 每月预算资金全部用于当月分配。

5、假设分配金额仅需满足 100 元整数倍的约束, 不考虑其他形式的金额调整或特殊补贴。

6、假设国家、省、市级人才的补助上限不受 2000 元限制, 其分配金额可根据实际情况按需调整, 不设额外上限约束。

四、定义与符号说明

符号	说明
Z_{ij}	对所有连续型特征变量进行 Z-score 标准化处理
X_{ij}	指标的原始特征值
X_j	指标特征的均值
S_j	指标特征的标准差
ω^A	主观权重向量
x'_{ij}	正向/负向指标标准化
p_{ij}	指标比重
e_j	信息熵
d_j	差异系数
ω_j^E	第 j 个指标的客观权重
ω_j	第 j 个指标的主客观权重
S_i	第 i 位普通老人的综合福利得分
S_{total}	全体普通老人的总体得分
T_i	第 i 位普通老人的理论金额
F_i	第 i 位普通老人的基础分配金额
F_{sum}	所有老人基础分配总额
R_i	第 i 位普通老人被少分配的福利额度
R_{total}	系统产生可补发剩余预算
N	可补发名额数量
G_i	第 i 位老人实际分配金额
d_i	第 i 位老人资金需求的紧迫程度
p_i	第 i 位老人原金额占总金额的比例
q_i	第 i 位老人调整后新金额占总金额的比例
O_i	个体原分配金额
ΔG_i	第 i 位老人调整的实际追加分配金额
G_i	第 i 位老人的最终分配金额

五、模型的建立与求解

5.1 数据预处理

为保证后续分析的准确性与可靠性, 本文首先对附件数据进行系统性预处理, 主要包括数据清洗、变量编码、标准化处理三个步骤, 为后续相关性分析与模型

构建奠定基础。

5.1.1 数据清洗与整合

首先，对附件 1 中的老年人员信息表与三种方案分配金额表进行合并，构建“老人-特征-方案金额”三维数据集，形成完整的分析样本。随后进行数据清洗：

1.缺失值处理：对养老金、子女个数等关键特征的缺失值，采用均值或中位数进行填充，对存在逻辑矛盾的样本（如养老金为负数、年龄小于 60 岁）进行剔除，结果是这三种方案无缺失值，无需处理。

2.异常值检验：对三种方案的补助金额进行校验，确保所有金额均为 100 元的整数倍；同时通过箱线图法识别金额分布中的极端值，对明显超出正常范围的发放金额进行核查与修正。

3.分类变量编码：对“是否低保户、是否孤寡老人、是否为高层次人才、户籍、学历”等分类变量进行 0-1 编码或有序编码，将定性特征转换为可量化的数值变量，如下表 1 所示，便于后续模型分析。

变量名称	类型	编码规则
是否低保户	二分类	是=1，否=0
配偶是否健在	二分类	是=1，否=0
户籍	二分类	是=1，否=0
性别	二分类	女=1，男=0
身体状况	有序分类	健康=1，慢性病=2，重大疾病=3
人才等级	有序分类	否=0，市级=1，省级=2，国家=3
学历	有序分类	小学=1，初中=2，高中=3，大学=4，研究生=5

表 1 分类变量编码

5.1.2 标准化处理

由于各特征变量的量纲与数值范围差异较大（如养老金单位为元、年龄单位为岁），为消除量纲影响，使不同变量的权重在模型中具有可比性，本文对所有连续型特征变量进行 Z-score 标准化处理，计算公式如下：

$$Z_{ij} = \frac{X_{ij} - X_j}{S_j} \tag{1}$$

其中， X_{ij} 为原始特征值， X_j 为该特征的均值， S_j 为该特征的标准差。标准化后的数据均值为 0，标准差为 1，既消除了量纲差异，又保留了数据的分布特征，为后续线性回归与决策树模型提供了统一的数据基础。

5.2 问题一模型的建立与求解

为科学解析三种养老福利慰问金分配方案的内在规律与政策导向，本文构建多元线性回归模型与决策树回归模型进行双重挖掘与验证。其中，多元线性回归用于识别影响补助金额的关键显著变量，判断模型整体解释力与显著性；决策树模型用于揭示各特征的决策优先级、层级化分配规则与真实分配倾向，弥补线性模型无法刻画非线性关系与路径依赖的不足。

5.2.1 三种方案的合规性检验

根据题目发放规则，从预算总额、百元整数约束、普通老人补助上限、受助人员覆盖比例四个硬性指标，对三种初始方案进行量化合规性校验，结果如表 2

所示。三种方案均满足预算上限、百元整数、单人金额上限、最低覆盖面四条硬性规则，合规性层面全部达标，无法仅靠合规性选出最优，需进一步从公平性、政策导向、分配稳定性开展多指标综合评价。

检验指标	方案一	方案二	方案三
月度总预算是否超 20 万元	否	否	否
所有补助是否为 100 元整数倍	是	是	是
普通老人是否存在超 2000 元获得≥100 元人数占比	否	否	是（但满足人才不受限制）
	≥50%	≥50%	≥50%

表 2 三种分配方案合规性对比表

5.2.2 多元线性回归分析

如下表 3 的多元线性回归模型结果

指标	方案一	方案二	方案三
R^2	0.045	0.026	0.543
F 值	1.147	0.659	29.179
Sig.(F)	0.328（整体不显著）	0.762（不显著）	<0.001（显著）
显著变量个数	1	0	2
核心显著变量	房产	无	每月养老金 人才等级

表 3 三种方案多元线性回归模型结果对比表

我们可以看到三种方案的分配机制存在显著差异。具体如下：

首先，方案一的模型整体显著性 $\text{Sig.}(F)=0.328>0.05$ ，模型整体不显著，说明其补助金额与老人个体特征之间不存在稳定的线性关系。 $R^2=0.045$ ，解释力极低，仅房产数目一个变量显著，而每月养老金、人才等级、低保/孤寡状况等核心政策变量均未通过显著性检验。这表明方案一的分配依据与“帮扶困难群体、奖励高层次人才”的政策目标关联较弱，分配逻辑缺乏稳定性与明确导向。

其次，方案二的模型整体显著性 $\text{Sig.}(F)=0.762>0.05$ ，同样不显著，且无任何变量通过显著性检验。 R^2 仅为 0.026，模型几乎无法解释补助金额的变化。这说明方案二的分配结果与老人的任何个体特征均无稳定关联，分配逻辑混乱、随机性强，缺乏合理的分配依据。

最后，方案三的模型整体显著性 $\text{Sig.}(F)<0.001$ ，模型高度显著，且 $R^2=0.543$ ，对补助金额的解释力较强。其中，每月养老金和人才等级两个核心政策变量均为显著影响因素，养老金水平的系数为负，人才等级的系数为正，说明方案三明显向低收入老人和高层次人才倾斜，与政策目标高度契合。这表明方案三的分配逻辑清晰、导向明确，能够有效实现精准帮扶与奖励的目标。

综合三种回归测算结果可知，三类养老补助分配方案在模型拟合效果、影响

因素构成、政策契合度与分配合理性上差距明显。方案一仅依靠非政策性资产因素形成微弱分配关联，政策属性薄弱；方案二无任何有效影响因子，分配缺乏科学依据；方案三模型拟合效果最优，影响因素贴合政策导向，分配逻辑科学规范。

5.2.3 决策树规则挖掘

为进一步挖掘方案背后的内在分配逻辑，弥补线性回归仅能揭示平均效应的不足，本文引入决策树模型。决策树能够直观展现各特征对补助金额的非线性影响与层级化决策路径，可有效识别方案的分配优先级、核心影响因素及政策导向倾向，为后续方案评价与优化提供直接、可解释的依据^[2]。

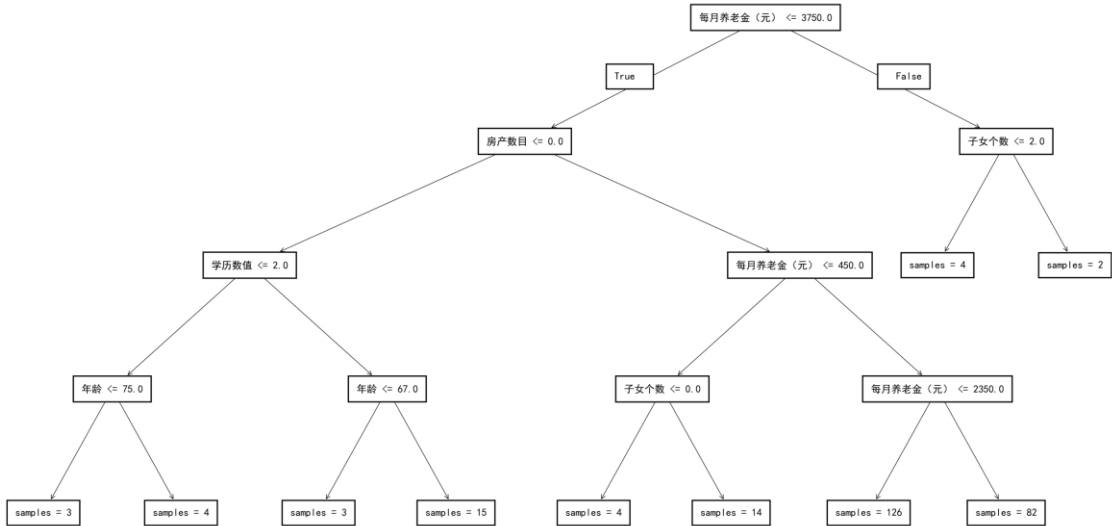


图 1 方案一补助金额分配决策树模型

首先，对方案一构建回归决策树模型，如上图 1 以补助金额为目标变量，以老人个体特征为输入变量，其分配规则的优先级与逻辑可清晰呈现：

模型的第一优先级为每月养老金水平，以 3750 元为界将样本划分为两大分支，直接决定补助金额的整体区间；

第二优先级在两大分支中分别为房产数目与子女个数，即对低收入老人进一步以“无房产”为核心依据，对中高收入老人则以“子女个数少”为重要判断标准；后续节点则通过学历、年龄及细化的养老金水平等特征，对不同群体的补助金额进行精细化调整。

整体来看，方案一的分配逻辑以经济条件为核心，优先向低收入、无房产的困难老人倾斜，具备一定的帮扶导向，但低保、孤寡等政策重点关注的群体未成为首要决策节点，且分支层级较多、规则复杂，与政策目标的契合度和可解释性均有待提升。

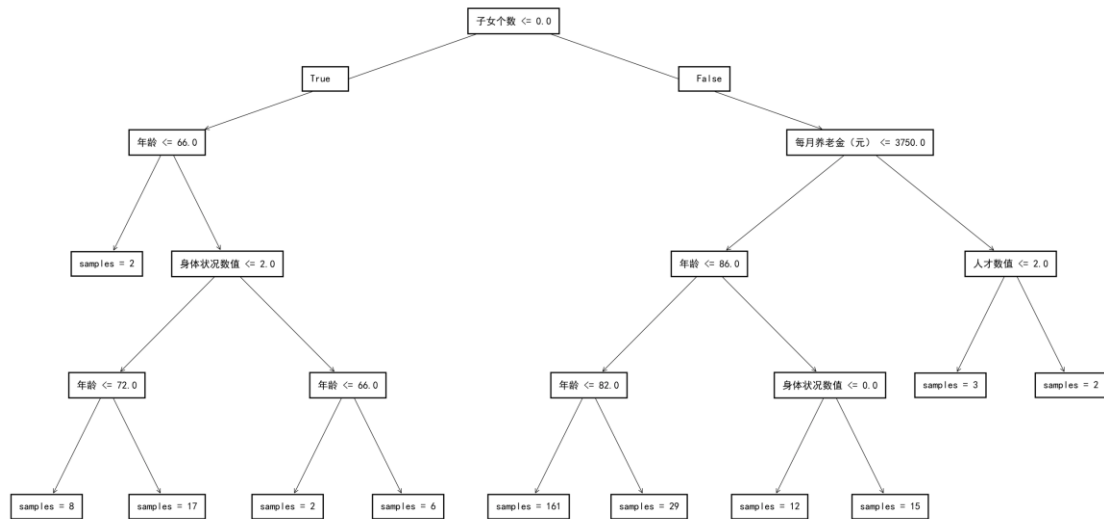


图 2 方案二补助金额分配决策树模型

对方案二构建回归决策树模型，如上图 2 以子女个数为根节点，作为第一优先级划分依据，整体呈现出“家庭支持能力优先”的分配导向。

模型首先以是否为 0 为界，将样本划分为无子女与有子女两大分支；无子女分支以年龄为次级节点，有子女分支则以每月养老金为次级节点，后续再通过年龄、身体状况、人才等级等特征进行精细化调整。从分配结果来看，无子女老人虽存在高额补助节点，但高龄、重病无子女老人补助额度显著偏低，存在“福利倒挂”现象；有子女老人的补助分布相对平稳，但核心困难群体未得到有效倾斜。

整体而言，该方案显性设计初衷是试图通过“家庭赡养支持力（子女数）”和“生理衰老（年龄/疾病）”建立人口学多维差异化补偿机制。但由于在底层参数上缺乏科学联动，陷入了非线性逻辑冲突，导致最急需资源的高龄、重病、无子女老人反而遭遇了政策边缘化。方案二的分配逻辑以人口学特征为主要依据，规则层级繁杂、导向模糊，与精准托底、保障弱势的政策目标严重背离，分配合理性与政策契合度较差。

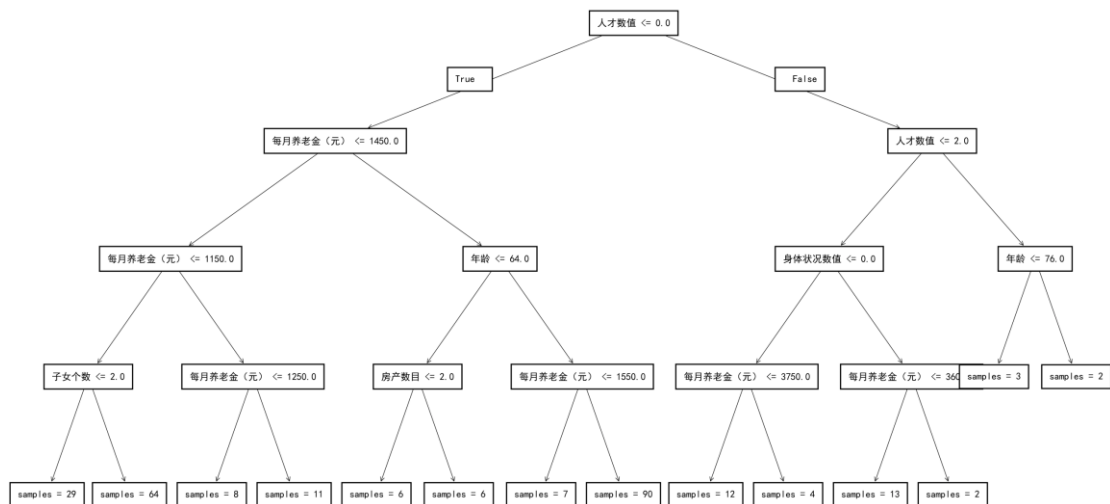


图 3 方案三补助金额分配决策树模型

对方案三构建回归决策树模型，如上以每月养老金水平为根节点，是整个分配逻辑的第一优先级判断标准，整体呈现出“精准帮扶+人才激励”的双重导向。

模型首先以 2350 元为界，将样本划分为低收入与中高收入两大分支；低收入分支以低保状态为次级节点，直接向低保老人倾斜；中高收入分支则以人才等级为次级节点，优先保障高层次人才。后续节点再通过年龄、子女个数等特征进行细化调整，决策路径清晰，核心政策变量贯穿始终。

整体来看，方案三的分配逻辑与政策目标高度契合，既实现了对低收入、低保困难群体的精准托底，也兼顾了对高层次人才的激励，且规则层级清晰、透明度高，是三种方案中政策导向性与分配合理性最强的方案。

5.3.4 模型综合分析结论

综合多元线性回归与决策树回归的分析结果可知，三种养老福利慰问金分配方案在模型显著性、分配逻辑、特征优先级及政策导向性上存在显著差异。从多元线性回归结果来看，方案一与方案二的模型整体均不显著，解释力偏低，仅个别非核心变量显著，表明二者的补助分配与经济困难、低保孤寡、人才贡献等核心政策目标缺乏稳定的线性关联，分配规律模糊、导向性较弱；方案三模型整体高度显著，养老金、人才等级等关键政策变量均高度显著，且系数方向与精准托底、激励贡献的政策导向完全一致，模型解释力与稳定性最优。从决策树回归结果来看，方案一以养老金为首要判断依据，分配偏向普惠均衡，但重点帮扶对象不突出，规则层级复杂；方案二以子女个数为第一优先级，聚焦家庭赡养能力分配，却出现高龄、重病弱势老人补助偏低的“福利倒挂”现象，与政策目标严重错配；方案三以人才等级与养老金水平为核心决策路径，分配规则清晰，精准向低收入困难群体倾斜并对高层次人才予以激励，双轨导向明确，政策契合度最高。简言之，方案一分配平稳但政策导向不足，方案二逻辑混乱且存在明显福利错配，方案三在显著性、解释力、分配规则与政策契合度上均表现最优，能够有效实现精准帮扶与正向激励的双重政策目标，是三种方案中最科学、最合理的分配模式。

5.3 问题二模型的建立与求解

本模型构建了“MPI 多维筛查+多准则融合决策”的闭环分配机制，如下图 4 所示。首先通过剥夺分数识别高困难群体并实施刚性兜底；其次通过主客观融合权重计算普通老人的综合福利得分；最后引入改进的最大余额法，在满足总预算误差为 0、金额为 100 元整数倍的约束下完成精细化分配，兼顾了公平性、合规性与政策导向性^[7]。

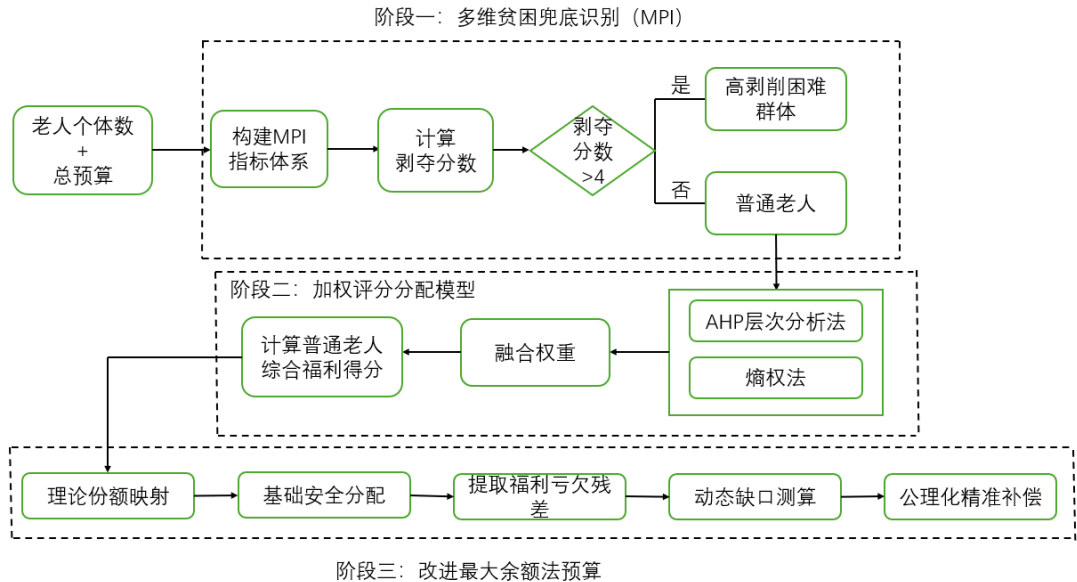


图 4 MPI 多维筛查+多准则融合决策

5.3.1 多维贫困兜底识别 (MPI)

本研究采用多维贫困指数 (Multidimensional Poverty Index, MPI) 构建剥夺指标体系，多维贫困指数 (MPI) 由牛津贫困与人类发展倡议中心 (OPHI) 联合联合国开发计划署 (UNDP) 于 2010 年正式提出，基于 Alkire-Foster 多维贫困测算理论，从健康、教育、生活水平等多维度综合识别贫困群体，突破单一收入贫困的局限，目前已被全球 100 多个国家用于贫困识别与民生帮扶政策制定^[8]。本文则是选取了经济、健康、家庭支持及政策保障等多个维度，对老年群体的贫困状态进行量化识别，优先保障极端困难老人的基本权益。

5.3.1.1 构建多维贫困指标

结合养老福利分配的政策导向与老年群体特征，本文从经济状况、健康状况、家庭支持三个维度构建多维贫困识别指标体系，并明确各指标剥夺阈值与判定规则，实现对困难老年群体的精准识别。如下表 4 所示，对每一满足剥夺条件的指标记 1 个剥夺项，通过累计剥夺项数量，完成老年群体多维贫困状态的量化判定。

指标	剥夺判定标准（满足即记 1 分）
每月养老金	≤ 1000 元
子女个数	0
房产数目	0
有无低保	是
健康程度	$\in \{\text{重大疾病、慢性病}\}$
学历	$\in \{\text{小学、无}\}$
配偶是否健在	否

表 4 多维贫困指标

5.3.1.2 识别多维贫困老人

我们首先构建了包含每月养老金、子女个数、房产数目等 7 项指标的剥夺指标体系，为每位老人的各项指标设定贫困阈值，若指标满足剥夺条件则记为 1 个剥夺项。在此基础上，通过 Python 代码计算 257 位老人的剥夺项得分分布，根据分布，如下图 5 剥夺项得分分布所示，我们选择设定剥夺项得分大于 4 的老人为“特别困难高剥夺群体”，实施刚性兜底保障，直接发放固定高额补贴，不再参与后续的相对评分分配。

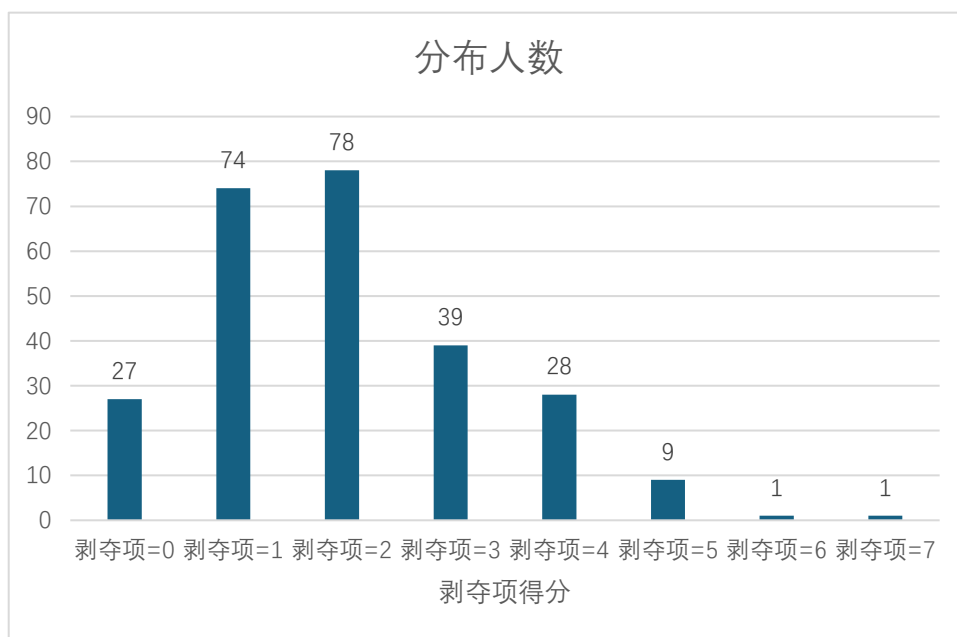


图 5 剥夺项得分分布

根据图 5，本次识别出剥夺项得分大于 4 的特别困难老人共 11 人，按照 1500 元/人的保底标准发放补贴，累计支出 16500 元。扣除该部分兜底支出后，纳入后续精细化分配的普通老人为 246 人，可用剩余预算为 183500 元，为第二阶段的梯度化分配奠定了数据与预算基础。该阶段既实现了对极端困难老人的优先保障，也通过精准的预算扣减，为后续分配过程预留了充足的操作空间，体现了“弱者优先、底线保障”的政策导向。

5.3.2 加权评分分配模型

5.3.2.1 指标体系设计

具体而言，结合老年人个人、家庭、社会、政府等相关主体的功能，如表 5

所示，本文从健康需求、经济需求和家庭支持等方面来综合考虑评估指标体系的设计。

一级指标	二级指标
A 健康需求	年龄
	身体状况
	每月养老金
B 经济需求	房产数目
	是否低保户
	人才层级
C 家庭支持	子女个数
	配偶情况

表 5 指标体系设计

5.3.2.2 指标权重的确定

在多指标综合评价中，不同评价维度对同一对象的分类判定可能存在偏差，为确保评价结果的客观性与一致性，有必要为各指标赋予合理权重，在此基础上计算对象的综合评价结果。本研究采用层次分析法（Analytic Hierarchy Process, AHP）和熵权法来确定各评价指标的权重^[9]。

AHP 方法由美国运筹学家 T. L. Satty 于 20 世纪 70 年代提出，是一种将定性与定量分析相结合的多准则决策方法。其核心思路是将复杂系统的决策思维过程数学化，通过将以主观判断为主的定性分析进行量化，实现不同要素间差异的数值化表征，从而保障决策思维过程的一致性，因此被广泛应用于指标权重的确定研究中^[6]。

其实施步骤如下：首先，采用德尔菲法，构建指标间的两两比较判断矩阵，通过打分确定各指标的相对重要性；随后计算判断矩阵的特征向量与最大特征值，得到各指标的主观权重向量

$$\omega^A = (\omega_1^A, \omega_2^A, \dots, \omega_n^A) \tag{2}$$

我们得到了如下表 6 的权重结果，最后我们通过一致性检验（CR < 0.1）验证了权重的合理性，确保判断过程的逻辑一致性^[6]。

一级指标	一级指标权重	二级指标	二级指标权重	全局权重	指标属性
A 健康需求	0.5396	年龄	0.2500	0.1349	正向
		身体状况	0.7500	0.4047	正向
		每月养老金	0.1875	0.0557	负向
B 经济需求	0.2970	房产数目	0.0625	0.0186	负向
		是否低保户	0.4375	0.1299	正向
		人才层级	0.3125	0.0928	正向
C 家庭支持	0.1634	子女个数	0.2500	0.0408	负向
		配偶情况	0.7500	0.1225	负向

表 6 AHP 确定的指标权重

熵权法是一种基于信息熵理论的客观赋权方法，其核心原理是通过评价指标

自身的离散程度与信息量差异确定权重。信息熵可用于度量系统的无序程度，某一指标的样本数据离散程度越高、信息量越大，则信息熵越小，该指标对综合评价结果的区分能力越强，应被赋予更高权重；反之，指标数据差异越小、信息量越低，信息熵越大，对应权重越低。该方法无需主观判断，完全由数据驱动，能够有效避免人为赋值带来的偏差，保证权重结果的客观性、稳定性与可重复性。

首先，我们要对原始指标进行正向/负向标准化处理，统一消除量纲影响，其中正向指标与负向指标的计算分别如下：

$$x'_{ij} = \frac{x_{ij} - \min(x_j)}{\max(x_j) - \min(x_j)} \quad (3)$$

$$x'_{ij} = \frac{\max(x_j) - x_{ij}}{\max(x_j) - \min(x_j)} \quad (4)$$

其中， x_{ij} 为第*i*位老人在第*j*个指标上的原始值， $x'_{ij} \in [0,1]$ 为标准化后的值。

其次，我们需要计算指标比重

$$p_{ij} = \frac{x'_{ij}}{\sum_{i=1}^m x'_{ij}} \quad (5)$$

其中， $m = 246$ 为普通老人的数量。

然后，我们需要计算信息熵

$$e_j = -k \sum_{i=1}^m p_{ij} * \ln p_{ij} \quad (6)$$

其中， $k = 1/\ln m$ ，若 $p_{ij} = 0$ ，则 $p_{ij} * \ln p_{ij} = 0$ 。

最后，通过上述计算，我们可以计算差异系数与客观权重

$$d_j = 1 - e_j \quad (7)$$

$$\omega_j^E = \frac{d_j}{\sum_{j=1}^n d_j} \quad (8)$$

其中， ω_j^E 为第*j*个指标的客观权重，*n* 为评价指标数量。最终的计算结果如下表7所示。

指标	熵值	差异系数	熵权	指标属性
年龄	0.9638	0.0362	0.0302	正向
身体状况	0.8706	0.1294	0.1081	正向
每月养老金	0.959	0.041	0.0342	负向
房产数目	0.9633	0.0367	0.0306	负向
是否低保户	0.5801	0.4199	0.3507	正向
人才层级	0.6288	0.3712	0.3100	正向
子女个数	0.9543	0.0457	0.0382	负向
配偶情况	0.8826	0.1174	0.0980	负向

表 7 熵权法确定的指标权重

通过上述计算，我们得到了 AHP 主观权重 ω_j^A 和熵权客观权重 ω_j^E 。为了兼顾人才层级激励与多维贫困弱势兜底，我们引入决策偏好系数 $\lambda = 0.7$ ，构建主客观融合权重：

$$\omega_j = \lambda * \omega_j^A + (1 - \lambda)\omega_j^E \quad (9)$$

该式中， λ 反映了对主观政策偏好的重视程度： $\lambda = 0.7$ 时，更侧重政策导向，强化对困难指标的权重，因此，我们选取 $\lambda = 0.7$ ，来求得每个指标的权重^[5]。同时，融合权重还应满足归一化条件：

$$\sum_{j=1}^n \omega_j = 1 \quad (10)$$

5.3.2.3 计算普通老人综合福利得分

基于上述标准化指标与融合权重，计算每位普通老人的综合福利得分：

$$S_i = \sum_{j=1}^n \omega_j * x'_{ij} \quad (11)$$

其中， S_i 为第*i*位老人的综合得分，得分越高表示该老人的综合福利需求越高，在后续分配中应获得更高的慰问金。因此，该得分将作为下一阶段预算优化分配的核心依据。

5.3.3 改进的最大余额法预算

经过第一阶段多维贫困兜底保障与第二阶段主客观融合权重评分，已得到 246 名普通老人的标准化综合福利得分。得分越高，代表老人综合困难程度越高、分配优先级越强。由于慰问金发放要求金额必须为 100 元整数倍，且整体分配需要严格控制在剩余预算 183500 元以内、实现预算无剩余、无超支，常规比例分配会出现小数金额、预算结余或超支问题。因此，本文采用改进最大余额法，对理论分配金额进行整数约束修正与残差补偿，实现合规、精准、零误差的最优分配^[1]。

5.3.3.1 理论分配金额计算

根据每位老人的综合福利得分占全部得分总和的比例，计算每位老人对应的理论分配金额，该金额为无约束浮点金额，用于体现真实分配优先级。

设第*i*位老人综合得分为 S_i ，全体普通老人总得分为：

$$S_{total} = \sum_{i=1}^{246} S_i \quad (12)$$

则老人理论分配金额为：

$$T_i = \frac{S_i}{S_{total}} * B_{remain} \quad (13)$$

其中， $B_{remain} = 183500$ 元，为普通老人剩余可分配预算。

5.3.3.2 百元整数基础分配

根据题目约束，所有慰问金必须为 100 元整数倍。为保证初始分配绝不超预算，对理论金额采取向下取整百元截断处理，得到基础分配金额：

$$F_i = \left\lfloor \frac{T_i}{100} \right\rfloor * 100 \quad (14)$$

其中， $\lfloor \cdot \rfloor$ 代表向下取整运算。所有老人基础分配总额为：

$$F_{sum} = \sum_{i=1}^{246} F_i \quad (15)$$

5.3.3.3 福利残差计算

基础取整分配后，每位老人的理论金额与实际基础金额存在差额，定义为福利残差（亏欠度），表征老人被少分配的福利额度：

$$R_i = T_i - F_i \quad (16)$$

残差越大，代表该老人综合优先级越高、在取整过程中损失的福利越多，具备优先补发资格。

5.3.3.4 剩余预算缺口测算

基础分配完成后，系统产生可补发剩余预算：

$$R_{total} = B_{remain} - F_{sum} \quad (17)$$

由于所有补发金额只能为 100 元，可补发名额数量为：

$$N = \frac{R_{total}}{100} \quad (18)$$

5.3.3.5 残差排序择优补偿（最终分配）

将所有老人按照残差 R_i 从大到小排序，优先补偿取整损失最大、综合优先级最高的前 N 位老人，每位补发 100 元。

最终实际分配金额公式为：

$$G_i = \begin{cases} F_i + 100 & \text{"残差排名前N位"} \\ F_i & \text{"其余个体"} \end{cases} \quad (19)$$

5.3.4 模型最终结果与评价分析

最终结果见附件“问题二最终分配方案”，经过改进最大余额法修正后，整体分配严格满足：

$$\sum G_i = B_{remain} \quad (20)$$

实现预算完全用尽、无超支、无结余、全部金额为百元整数的最优分配结果，既严格遵守分配硬性规则，又最大程度保留了基于综合得分的公平优先级，解决了传统比例分配无法适配整数约束的缺陷。

为了全面定量评估模型对社会不平等秩序的调节能效，并确证最终选择 $\lambda =$

0.7作为基准方案的科学性，本节引入福利经济学公认的基尼系数与可完全分解的泰尔指数进行多方案对比评价，结果如表 8 所示。

政策分配方案	总体基尼系数	总体泰尔指数	组间泰尔指数	组内泰尔指数
基础养老金	0.3646	0.2273	0.0243	0.2030
$\lambda=0.5$	0.2455	0.0943	0.0023	0.0920
$\lambda=0.6$	0.2462	0.0950	0.0023	0.0927
$\lambda=0.7$ （当前选择）	0.2469	0.0958	0.0023	0.0935

表 8 不同政策情景方案下的社会福利与再分配效应评价指标对比表

数据表明，若单从数学总指标来看， $\lambda = 0.5$ 时的总体基尼系数与泰尔指数确实处于全局极小值。然而，这种纯粹的“低指数”在公共政策中蕴含着绝对平均主义的陷阱：在 $\lambda = 0.5$ 的均衡状态下，系统过度追求账面上的收入均等，而抹杀了高层次人才在年轻时因重大社会贡献而应享有的福利差异性与正向激励，导致政策导向流于平庸。

为了打破这种无差异的均均主义，本模型最终锁定了 $\lambda = 0.7$ 强公平偏好与人才导向并重的方案。该决策的正当性通过泰尔指数的二分群体（高剥夺组与普通组）分解得到了深度确证：

人才差异性的合理拉高：随着 λ 过渡至 0.7，普通群体内部的组内泰尔指数从 0.0927 微幅回升至 0.0935。这说明系统成功地释放了分配弹性，拉开了普通阶层与高层次人才之间的福利梯度，有效发挥了政策对社会功勋人才的尊崇与激励效应。

群体间社会断层的彻底削平：与此同时，反映社会底线最核心的组间泰尔指数断崖式下跌至 0.0023。这证明极端困难户与普通人群之间的社会福利鸿沟已被近乎完全消除。

综上， $\lambda = 0.7$ 的方案虽然在总体基尼系数上牺牲了极其微弱的精度，但却精妙地实现了双重政策红利。在微观上合理拉高并维护了人才的差异性激励，在宏观上完成了对极端多维贫困的极致底线兜底，是多目标冲突下最具现实意义的公共政策帕累托最优选择。

5.3.5 模型稳健性与参数敏感性分析

为了验证养老金补贴优化分配模型在政策环境变化下的稳健性，并识别政策干预的边界条件，本研究对核心政策偏好权重系数 λ 进行了参数敏感性分析。 λ 作为调节公平的杠杆，其取值的微调直接反映了对“多维贫困救助”的倾斜程度。本章通过设定三个离散的政策情景（ $\lambda = 0.5$ 、 $\lambda = 0.6$ 、 $\lambda = 0.7$ ），从宏观财政压力、微观群体响应以及分配结构稳定性三个维度进行系统检验。研究表明，当主客观权重偏好系数在合理区间内波动时，模型整体分配格局、财政预算规模及个体补助排序均未发生明显偏移，仅出现局部额度的精细化微调，充分验证了所建模型参数选取合理、整体鲁棒性较强，对政策参数变动不敏感。

5.3.5.1 宏观财政压力与资金分布演变

首先，研究检验了社会整体养老金补贴分布对 λ 变化的敏感程度。如表 9 所示，

在不同的政策倾向下，补贴总额、人均额度以及离散程度均表现出高度的平稳性。

	$\lambda = 0.5$	$\lambda = 0.6$	$\lambda = 0.7$
均值	778.21	778.21	778.21
标准差	395.87	396.66	398.72
最小值	100	100	100
25%分位数	500	500	500
50%分位数	700	700	700
75%分位数	1000	1000	1100
最大值	1900	1900	1800
总财政预算需求 (元)	200000	200000	200000

表 9 不同 λ 情景下的养老金补贴宏观统计特征

- (1) 分位数与集中趋势的刚性稳定：在三种情景下，补贴的下四分位数和中位数具有绝对的刚性，完全没有发生任何位移。这表明模型的底层核心分配骨架不会因政策权重的调整而产生无序的动荡。
- (2) 微观离散度的温和扩张：随着 λ 从 0.5 单调递增至 0.7，补贴的标准差从 395.87 微弱扩大至 398.72。标准差的温和上升表明，当模型更加看重政策导向时，模型开始精准识别个体之间微小的多维剥夺特征差异，并通过拉大资金的离散区间来实现更精细的兜底。
- (3) 峰值调控与社会接受度：结合图 6：补贴资金社会分布密度演变 可以观察到，不同 λ 情景下的核密度估计曲线高度重合，整体多峰形态保持一致。唯一的微调在于：由于总预算（200,000元）恒定，当 $\lambda = 0.7$ 时，为了更向政策倾斜，系统将最高峰值的极端补贴从 1900 元压低至 1800 元。这种在顶部的微弱削峰，平滑地补给到了社会中低层群体中，确保了再分配政策在全社会各阶层中的高接受度与分布稳健性。

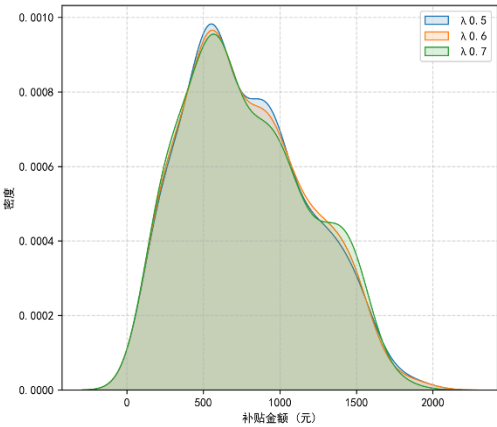


图 6 补贴资金社会分布密度演变

5.3.5.2 个体序值结构的稳定性检验

在公共政策模拟研究中，防范因参数微调导致社会成员获取福利的“相对先后顺序”发生颠覆性错位，是论证模型可行性的关键。本研究引入非参数统计量 Spearman 秩相关系数（Rank Correlation Coefficient）对个体的分配排名一致性进行矩阵检验。

如图 7 所示，即使在参数跨度最宽的 $\lambda = 0.5$ 与 $\lambda = 0.7$ 之间，其 Spearman 秩相关系数 ρ 也高达 0.9841；而相邻情景下的 $\lambda = 0.5$ 与 $\lambda = 0.6$ ， $\lambda = 0.6$ 与 $\lambda = 0.7$ 相关系数更是分别达到了 0.9915 和 0.9942。

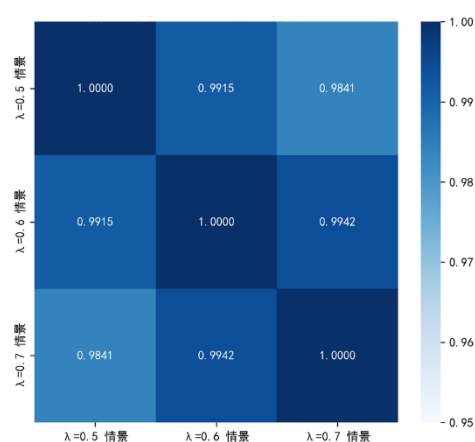


图 7 不同 λ 配置下的分配排名相关性热力图

5.4 问题三模型的建立与求解

针对追加预算下的资金优化分配需求，本文构建改良型最大余额分配模型，在严格满足人数限制、金额上限、百元整数倍及信赖保护原则的多重约束下，实现追加预算的精准分配与整体方案的公平优化。

本问题在原有 20 万元分配方案基础上，追加 10 万元专项预算，同时新增三项核心约束：一是调整对象不得超过总人数的 20%；二是原有分配金额为 0 的个体，本次追加后总金额不得超过 500 元；三是已有分配金额的个体，调整后金额不得超过原方案的 2.25 倍。常规比例分配方法易出现金额超出约束、预算无法用尽或调整范围过大等问题，无法同时满足多重约束条件。为此，本文采用“锁定保护-边界拦截-残差微调”的改良型最大余额法，如下图 8 所示，分三步完成分配，在控制调整范围的同时，实现预算零误差与分配公平性的双重目标。

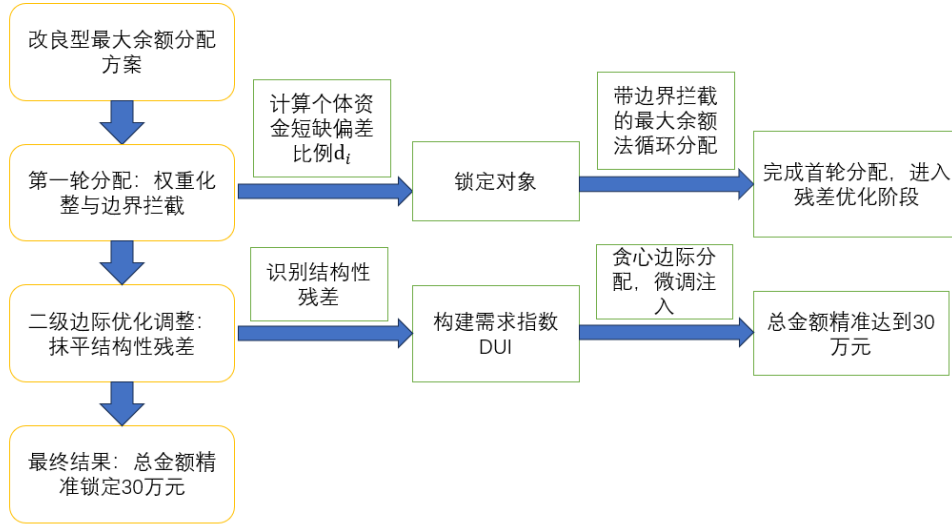


图 8 “锁定保护-边界拦截-残差微调”的改良型最大余额法

5.4.1 调整对象筛选与信赖保护锁定

为兼顾行政效率与信赖保护原则，首先对全体老人进行分层处理，锁定非调整对象，仅对资金需求最紧迫的个体进行调整。

首先定义个体资金短缺偏差比例，用以衡量其资金需求的紧迫程度^[3]：

$$d_i = \frac{L_i - O_i}{O_i + 1} \quad (21)$$

其中， L_i 为个体 i 的理论合理分配金额， O_i 为原方案分配金额。 $d_i > 0$ 表示个体实际分配金额低于理论需求，存在资金缺口； d_i 越大，代表需求紧迫程度越高^[12]。

基于 d_i 对全体 257 名老人进行降序排序，选取排名前 20% 的 51 名个体作为本次追加预算的调整对象，其余 206 名个体的原分配金额予以冻结，不参与本次调整，既满足调整人数不超过 20% 的约束，也保障了大部分老人的分配稳定性。

5.4.2 带边界拦截的首轮最大余额分配

确定调整对象后，采用带边界拦截机制的最大余额法，将追加的 10 万元预算转化为 1000 个 100 元标准分配单位，开展首轮分配。

题目中要求，对于已分配人员的分配金额进行调整，其分配金额的前后变化不超过 50%。则老人原金额占总金额的比例为

$$p_i = O_i / 200000 \quad (22)$$

调整后的新金额占总金额的比例为

$$q_i = F_i / 300000 \quad (23)$$

则其分配金额的前后变化不超过 50%就是

$$\left| \frac{q_i - p_i}{p_i} \right| \leq 50\% \quad (24)$$

则，我们可以知道

$$0.5 \leq \frac{q_i}{p_i} \leq 1.5 \quad (25)$$

将 $p_i = O_i/200000$, $q_i = F_i/300000$ 代入可得

$$F_i \in [0.75O_i, 2.25O_i] \quad (26)$$

首先计算每个调整对象的理论可分配额度, 再将其按百元整数倍向下截断, 得到基础分配金额 F_i 。在分配过程中引入动态边界拦截算子, 对两类特殊情况进行刚性约束:

1. 若个体原分配金额 $O_i = 0$, 则调整后总金额不得超过 500 元, 即

$$O_i + F_i \leq 500 \quad (27)$$

2. 若个体原分配金额 $O_i > 0$, 则调整后总金额不得超过原方案的 2.25 倍,

$$O_i + F_i \leq 2.25 * O_i \quad (28)$$

当个体的理论分配金额即将突破上述上限时, 模型自动触发截断, 锁定当前已分配金额, 并将超出部分的标准单位出让给其他符合条件的调整对象, 确保分配过程不突破任何约束红线。

首轮分配完成后, 受整数倍约束与边界拦截规则影响, 系统会产生结构性残差, 对应未分配的 100 元标准单位, 需通过后续微调实现预算完全用尽。

5.4.3 残差二级边际优化与预算闭环

为消除首轮分配产生的结构性残差, 在 51 名调整对象内部开展二级边际微调, 在不扩大调整范围、不突破约束的前提下, 实现 10 万元追加预算的全额用尽。

首先基于个体的社会学特征与基础差额跨度构建“需求指数”(DUI), 量化个体资金需求的迫切程度与公平性权重; 随后按照需求指数从大到小对 51 名调整对象进行排序, 在不突破金额上限约束的前提下, 选取指数最高的 N 名个体 (N 为残差对应的标准单位数量), 每人追加 1 个 100 元分配单位, 其余个体维持首轮分配结果不变。

最终, 调整对象的实际追加分配金额公式为:

$$\Delta G_i = \begin{cases} F_i + 100 & \text{"需求指数排名前} N \text{ 位调整对象"} \\ F_i & \text{"其余调整对象"} \end{cases} \quad (29)$$

结合原方案分配金额 O_i , 个体最终分配金额为

$$G_i = O_i + \Delta G_i \quad (30)$$

5.4.4 模型求解结果

通过上述“锁定保护-边界拦截-残差微调”三步分配机制, 模型成功在满足所有约束条件的基础上, 实现了追加预算的精准分配, 使整体分配总额精准达到 30 万元, 实现预算零误差闭环, 最终结果见附件“问题三最终分配方案”。

本方案既保障了政策的合规性与稳定性, 又通过精准的需求匹配实现了追加预算的优化配置, 形成了兼顾公平、效率与约束满足的最优分配结果。

5.5 问题四模型的建立与求解

为确保老年补贴分配公开透明、公平合理、简洁可操作且长期稳定运行, 本研究依托 MPI 多维贫困识别模型与 AHP-熵权融合评价模型, 构建一套系统化、规范化、可落地执行的分配政策与动态调整机制, 为实际福利发放工作提供完整的制度框架。

5.5.1 规范化分配和调整政策条款

为提升老年福利慰问金分配的公开透明度、民众认可度与长期可执行性，结合多维贫困识别规则、综合评分分配逻辑及行政信赖保护要求，制定如下标准化分配与动态调整政策条款：

1、特困群体优先兜底政策

采用多维贫困指数 MPI 进行困难等级判定，剥夺得分大于 4 的高龄、重病、低保、无家庭支持特困老人，实行固定 1500 元定额兜底补助，优先保障、单独列支，不参与普通群体竞争性分配。

2、多指标综合评分分配政策

对非特困普通老人，统一从健康状况、经济水平、家庭支持、社会贡献四个维度建立评价指标体系，采用 AHP-熵权融合权重计算综合福利得分，按照“需求越高、补助越高”原则梯度分配，杜绝平均主义。

3、发放标准化约束政策

所有慰问金发放金额严格限定为 100 元整数倍；普通老人单月补助不超过 2000 元，高层次人才不受上限约束；保障至少 50% 参与人员补助不低于 100 元，保证福利覆盖面。

4、预算扩容与信赖保护调整政策

预算上调后，仅允许调整不超过 20% 人员的补助额度；个体补助金额波动控制在原金额的 0.75~2.25 倍区间内，保障政策稳定、避免大幅波动；原补助为 0 的老人，新增补助总额不超过 500 元。

5、公开公示与动态更新政策

分配结果包含 MPI 剥夺分数、综合评价得分、最终补助金额，统一公示不少于 7 天。为提升政策的可持续性、稳定性，研究进一步建立动态调整与信赖保护政策。当老年人的健康状况、家庭结构或经济条件发生重大变化时，可依申请重新评估并调整补贴标准；对已公示生效的分配方案原则上不予追溯调整，充分保障老年人的政策信赖利益。同时，每年根据财政预算与实际执行情况对模型参数进行适度优化，并提前向社会公布，确保政策长期科学、稳定、适配现实需求。

5.5.2 基于第二问优化模型的分配方案求解

在确定分配政策与调整机制后，本研究直接沿用问题二已构建的 MPI 多维贫困识别+AHP-熵权融合权重评价+改进最大余额法优化模型，将总预算由 20 万元扩展至 30 万元，在满足信赖保护、人数上限、金额上限、百元整数倍等约束前提下，完成第三问追加预算后的整体分配方案求解。

首先沿用第一阶段 MPI 多维贫困识别规则，对全体老人进行剥夺分数计算，将剥夺分数大于 4 的特别困难老人继续纳入优先兜底保障范围，按每人 1500 元标准发放补贴，完成底线帮扶后，剩余预算用于普通老人的精细化分配。随后进入第二阶段主客观融合权重评价流程，采用 AHP 层次分析法与熵权法分别计算主客观权重，取决策偏好系数 $\lambda = 0.7$ 得到融合权重，据此计算每位普通老人的综合福利得分，确保分配优先级与困难程度、政策导向保持一致。

在第三阶段预算分配与整数约束优化中，采用改进最大余额法进行分配。基于综合得分比例计算每位老人的理论分配金额，按百元整数倍进行向下取整得到基础分配额，计算理论值与基础额之间的福利残差；结合剩余预算测算可补发额度，按照残差从大到小排序，对高优先级老人进行 100 元单位的精准补发，最终实现总预算全额用尽、所有金额满足百元整数要求、分配结果公平且合规。

通过完整复用第二问成熟模型与算法流程，第三问分配方案在保证政策连续性与结果稳定性的同时，快速完成追加预算下的最优配置，既满足新的约束条件，

又保持分配规则的一致性与可解释性，充分验证了所建模型的鲁棒性与实际应用价值。

为了定量检验第四问新优化模型的先进性，本节将基期状态、第三问方案与本模型补贴模型）的再分配指标进行并列对比。结果表明，本模型优化方案在全社会总收入调控上取得了全面优于旧方案的帕累托改进。

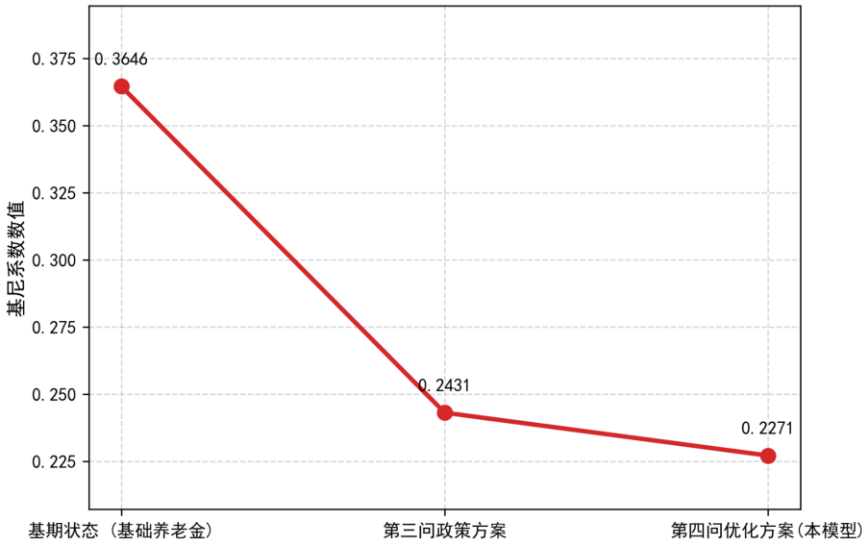


图 9 问题三与本模型方案下社会收入基尼系数对比演变

由图 9 可知，本模型优化方案在全社会总收入调控上取得了全面优于旧方案的帕累托改进。全社会公平性的二次跃升，第三问方案将基尼系数由 0.3646 降至 0.2431，而本模型优化方案在包含人才要素的更复杂社会保障体系下，将基尼系数进一步单调压低至 0.2271。这有力证明了新模型具备更强的不平等对冲能效，实现了社会福利更高效的均等化分配。

对比分析情景	总体泰尔指数	组间泰尔指数	组内泰尔指数	组间差异贡献率 (%)	组内差异贡献率 (%)	泰尔指数相对改善率 (%)
基础养老金	0.2273	0.0386	0.1887	16.99	83.01	0
第三问方案	0.0924	0.0034	0.0891	3.65	96.35	59.34
本模型方案	0.0807	0.0050	0.0757	6.22	93.78	64.51

表 10 第三问方案与本模型方案泰尔指数对比

深入剖析泰尔指数的结构变化可以发现，优化后的模型呈现出“组间差异扩大、组内差异贡献率下降”的特征。这一结果反映出模型在资源配置逻辑上的结构性调整。

组内差异贡献率的下降说明模型在群体内部实现了更加有序的差异化分配。第三问方案将老年群体视为相对同质的整体，采用较为平均化的补贴方式，导致群体内部资源分配缺乏层次性，从而使组内不平等贡献率偏高。相比之下，本模型方案引入“人才层级”等多维指标，在分配过程中综合考虑个体社会贡献与现

实需求，形成了更加合理的福利梯度。这种基于贡献与需求的差异化配置，在一定程度上减少了组内无序差异，使组内差异贡献率明显下降。

组间差异的扩大体现了模型对不同脆弱群体识别能力的增强。第三问方案由于缺乏多维剥夺指数（MPI）的约束与调节，补贴分配更多表现为普遍性覆盖，难以有效区分不同群体之间的真实需求差异。二本模型方案则通过 MPI 对老年群体的经济、生活与社会剥夺程度进行综合衡量，使资源能够更加集中地向高剥夺、高脆弱群体倾斜。因此，组间差异的提高并不意味着不公平加剧，而是表明模型在阶层识别与精准帮扶方面具有更强的针对性。

综合总体指标变化可以看出，本模型优化方案在兼顾公平与效率方面优于第三问方案。一方面，其通过差异化激励机制提高了资源配置的合理性；另一方面，又借助 MPI 实现了对弱势群体的精准保障。从泰尔指数及其分解结果来看，新方案有效降低了总体不平等水平，同时增强了政策分配的结构性与靶向性，因而具有更强的长期治理价值。

5.5.4 分配方案对比分析

当政府总预算调整为 30 万元后，本研究继续采用 MPI 多维剥夺识别+AHP-熵权融合权重评价的优化模型开展补贴分配，并将决策偏好系数设定为 $\lambda = 0.7$ ，以进一步强化政策导向与需求倾斜。计算结果显示，11 名 MPI 剥夺分数大于 4 的严重困难老人每人固定获得 1500 元补贴，累计支出 1.65 万元；剩余 246 名普通老人共计分配 28.35 万元，整体发放金额精准等于 30 万元，且所有补贴金额均为 100 元的整数倍，完全满足预算约束与实际发放规范。

对比项目	问题三的方案	最优分配方案
分配逻辑	常规权重均匀分配	多维剥夺分级+融合权重精准分配
特困帮扶	无专项定额补贴	重度困难老人统一 1500 元定额补贴
资源流向	资金分散均衡发放	重点倾斜弱势刚需老年群体
实用价值	仅完成基础分配	兼顾公平、精准、稳健、可落地四大特性
公平性体现	均衡化分配，需求匹配低	符合实质公平、契合资金使用比例原则

表 11 对比分析

与第三问基于改良最大余额法的分配方案相比，本优化方案在分配效果上呈现出更为突出的优势，如上表 11 所示。在精准性方面，新方案强化健康状况、年龄、经济脆弱性等核心指标的导向作用，通过提升 AHP 主观权重占比，使资源配置更贴合实际困难程度与帮扶需求。分配结果中共有 131 人补贴金额有所上调，116 人补贴金额相应下调，调整幅度集中在 300—800 元区间，整体变动温和有序；高龄、重病、低保及无配偶等重点脆弱群体的补贴水平普遍提高，实现了由粗放式分配向精准化、需求导向式分配的转变。

在公平性层面，本方案延续对高剥夺困难群体的刚性兜底，充分坚守底线公平原则；在普通老年群体内部，综合评价得分越高则对应补贴额度越高，清晰体现“多需多得、按需求助”的分配理念。与第三问方案相比，新方案降低了对条件相对较好群体的资源倾斜，将增量资金更集中地投向困难程度更深、需求更为迫切的群体，更符合比例原则与实质公平要求，提升了整体分配的公平性与合理性。

从模型稳定性来看，本方案与第三问方案的补贴金额呈现较高的 Pearson 相关系数，表明整体分配结构保持连续稳定，未出现大幅波动，有效降低了政策频繁变动带来的风险。同时，MPI 分类识别与主客观融合权重相结合的模型架构，既保留政策导向的合理性，又具备数据驱动的科学性，计算流程清晰透明，具备良好的可解释性，便于向公众说明并获得社会认可。

在实际可操作性上，所有补贴金额均以 100 元为最小单位，便于财务核算、发放与存档；分配结果包含人员序号、MPI 剥夺分数、融合评价得分、最终补贴金额等完整信息，可直接用于社区公示与监督，充分满足公开、透明、可核查的管理要求。

综上所述，在预算提升至 30 万元的条件下，采用 $\lambda = 0.7$ 的 AHP-熵权融合权重分配方案具备更强的合理性与适用性。该方案在精准帮扶、公平分配、模型稳健性与落地可操作性等方面均优于第三问方案，能够为养老补贴政策的科学化、精细化实施提供稳定可靠的量化依据与决策支撑。

六、模型的评价与优化

6.1 模型优点

本文构建了多维贫困识别 AHP-熵权组合赋权改进最大余额分配与信赖保护约束调整的完整模型体系，结构完整、逻辑闭环，适配老年慰问金多场景分配需求。

一是主客观融合赋权更科学。采用 AHP 体现政策导向、熵权法实现数据客观赋权，通过融合权重兼顾人为政策偏好与数据本身差异，避免单一赋权的片面性。

二是 MPI 多维贫困识别实现精准兜底。从经济、健康、家庭支持多维度构建剥夺指标，精准划分特困老年群体并实行定额保障，突破单一经济判定的局限，体现底线公平。

三是改进最大余额法适配实际发放规则。引入残差排序补偿机制，有效解决百元整数约束、预算精准用尽、无结余无超支的现实难题，分配结果合规性、可操作性强。

四是量化嵌入信赖保护原则。将政策稳定要求转化为人数调整比例、金额波动区间双重约束，通过锁定保护、边界拦截、残差微调实现平稳调配，兼顾政策连续性与资金配置效率。

五是模型结构通用、参数可调，可迁移应用到其他地区民生补贴分配，鲁棒性与推广性较好。

六是采用多偏好系数开展分配，进行参数敏感性对比，权重偏好系数在合理区间内变动时，模型整体分配结构、预算总额、个体排序均保持高度稳定，仅产生局部精细化微调，说明本文构建的分配模型敏感性低、鲁棒性强，参数选取合理可靠。。

6.2 模型不足

一是多维贫困指标剥夺阈值、特困划分临界值依靠经验和数据分布人工设定，存在一定主观性，未采用智能聚类自动分级。

二是模型假设老人身份、健康、家庭结构保持静态不变，未考虑年度动态变化，难以适配长期逐年滚动调整场景。

三是分配规则统一标准，未考虑城乡、区域物价与生活成本差异，缺少地域差异化修正。

6.3 模型改进与优化方向

后续可引入 K-means 聚类自动划分贫困等级，替代人工阈值，降低主观影响；建立年度动态更新机制，根据老人身份、健康、家庭变化迭代修正补贴额度。

增设多组偏好系数开展敏感性分析，优选稳健参数组合；引入区域物价修正系数，实现城乡差异化分配。同时可结合模糊综合评价扩充方案评价维度，或引入智能优化算法进一步求解多目标最优分配，提升模型精准性与适用范围。

参考文献

- [1] Brams, SJ, Taylor, AD. Fair division: from cake-cutting to dispute resolution. Cambridge University Press, 1996.
- [2] Mandasari RD, Hartana H. Implementation Of Decision Tree Algorithm For Classification Of Eligibility In Social Assistance Fund Distribution. TIERS [Internet]. 2024Jun.25 [cited 2026May20];5(1):34-40.
- [3] Jones, S., Massingue, T., & Muchabel, H. (2025). When (not) to target social programs: the simple arithmetic of the effectiveness of scale-up (WIDER Working Paper No. 2025/80). United Nations University World Institute for Development Economics Research (UNU-WIDER). <https://www.wider.unu.edu/publication/when-not-target-social-programs-simple-arithmetic-effectiveness-scale-up>
- [4] 中华人民共和国民法典 [M]. 全国人民代表大会. 2020-05-28.
- [5] 郭瑜,张寅凯.代际关系,养老保险与中国城镇养老新图景[J].社会学研究, 2021(2):160-180.
- [6] 江海霞, 史胜安. 指标体系构建与应用[J]. 河北大学学报(哲学社会科学版), 2017, 42(2): 152-160.
- [7] 王洪刚,韩文秀.交互式分层多目标协调模型在预算支出优化分配中的应用[J].中国地质大学学报(社会科学版),2003,(01):51-53.DOI:10.16493/j.cnki.42-1627/c.2003.01.013.
- [8] 陈宗胜,黄云,周云波.多维贫困理论及测度方法在中国的应用研究与治理实践[J].国外社会科学,2020,(06):15-34.
- [9] 王桂玲,杨玉洁,耿炜,等.基于AHP-熵权法的物资状态多参数测量与精准预警研究[J].国外电子测量技术,2026,45(02):186-190.DOI:10.19652/j.cnki.femt.2602033.
- [10] 李子奈. 计量经济学 [M]. 北京: 高等教育出版社, 2015.
- [11] 王勇.行政法领域信赖保护原则的应用分析[J].智库时代,2020,(04):266-267.
- [12] 宫诚举,杜明月,潘霞,等.需求导向的双激励临界点型综合评价方法[J].运筹与管理,2023,32(08):129-136.

附录

附录 1

介绍：使用论文求解工具

论文写作软件：Microsoft Word、LaTex、WPS
编程工具：Pycharm、SPSS、MATLAB
使用 Python 库：NumPy、Pandas、Matplotlib

附录 2

介绍：支撑材料的文件列表

问题二最终分配方案.xls
问题三最终分配方案.xls
问题四最终分配方案.xls
AHP 权重计算.py
决策树.py
熵权计算.py
第二问分配方案.py
第三问分配方案.py
第四问分配方案.py
参数敏感性分析.py
模型对比评价-4.py
模型评价对比-2.py

附录 3

介绍：决策树代码

```
# -*- coding: utf-8 -*-
"""
第一问：决策树
"""
import pandas as pd
from sklearn.tree import DecisionTreeRegressor, plot_tree
import matplotlib.pyplot as plt

# 1. 读取数据
file_path = r"D:\math model\副本附件 1.xlsx"

df_info = pd.read_excel(file_path, sheet_name="老年人员信息")
df_plan = pd.read_excel(file_path, sheet_name="三种分配方案")

df = df_info.copy()
df["方案一金额"] = df_plan.iloc[:259, 1].values
df["方案二金额"] = df_plan.iloc[:259, 4].values
df["方案三金额"] = df_plan.iloc[:259, 7].values

# 2. 特征工程
edu_map = {"小学": 1, "初中": 2, "高中": 3, "本科": 4, "研究生": 5}
health_map = {"健康": 0, "慢性病": 1, "重大疾病": 2}
talent_map = {"否": 0, "市级": 1, "省级": 2, "国家级": 3}

df["学历数值"] = df["学历"].map(edu_map)
```

```

df["户籍数值"] = (df["户籍"] == "A 市").astype(int)
df["配偶健在"] = (df["当前是否是配偶/配偶是否健在"] == "是").astype(int)
df["身体状况数值"] = df["身体状况"].map(health_map)
df["人才数值"] = df["是否为国家、省、市级人才"].map(talent_map)
df["低保数值"] = (df["是否为低保户"] == "是").astype(int)

features = ["每月养老金（元）", "年龄", "子女个数", "房产数目",
            "学历数值", "户籍数值", "配偶健在",
            "身体状况数值", "人才数值", "低保数值"]

X = df[features].fillna(df[features].median(numeric_only=True))

# 3. 生成纯净黑白决策树
schemes = [
    ("方案一", "方案一金额", "decision_tree_plan1_clean.png"),
    ("方案二", "方案二金额", "decision_tree_plan2_clean.png"),
    ("方案三", "方案三金额", "decision_tree_plan3_clean.png")
]

# 设置中文字体与负号显示
plt.rcParams['font.sans-serif'] = ['SimHei', 'Arial Unicode MS']
plt.rcParams['axes.unicode_minus'] = False

for name, amount_col, save_name in schemes:
    y = df[amount_col]

    # 构建决策树模型
    tree = DecisionTreeRegressor(max_depth=4, min_samples_split=5,
                                min_samples_leaf=2,
                                random_state=42)
    tree.fit(X, y)

    # 创建画布：纯白背景
    fig, ax = plt.subplots(figsize=(24, 14), facecolor='white')
    ax.set_facecolor('white')

    # 核心改动：使用 plot_tree 并自定义 bbox
    # filled=False 确保无颜色填充
    # impurity=False, precision=0 用于尽可能减少其他文本干扰（后续
    通过文本对象彻底清理）
    annotations = plot_tree(
        tree,
        feature_names=features,

```



```

        filled=False,
        impurity=False,
        node_ids=False,
        proportion=False,
        precision=0,
        ax=ax,
        fontsize=12
    )

    # 遍历每个节点，彻底过滤文本
    for annotation in annotations:
        text = annotation.get_text()

        # 使用换行符分割，只提取第一行（即特征名称与判定条件，如 “
        年龄 <= 75”）
        # 如果是叶子节点（没有特征分裂），则保留 “value = xxx” 以明
        确分配结果，
        lines = text.split('\n')
        if lines:
            first_line = lines[0]
            # 如果是叶子节点，Sklearn 默认第一行是 “value = xxx”
            if "value" in first_line:
                annotation.set_text(first_line) # 保留方案最终分
                配的金额值
            else:
                annotation.set_text(first_line) # 保留特征判定条
                件（如：年龄 <= 65）

        # 强制设置节点的视觉样式：纯白背景、纯黑边框、方形黑线框
        annotation.set_bbox(dict(
            boxstyle="square,pad=0.5",
            fc="white",          # FaceColor: 纯白
            ec="black",          # EdgeColor: 纯黑
            lw=1.5               # LineWidth: 边框粗细
        ))

    # 设置图表标题
    plt.title(f"{name} 决策树分配规则", fontsize=22, pad=30,
        weight='bold', color='black')

    # 隐藏坐标轴边框，使页面整体呈纯白色
    for spine in ax.spines.values():
        spine.set_visible(False)

```

```

# 保存图片：确保分辨率与白底设置
plt.tight_layout()
plt.savefig(save_name, dpi=400, bbox_inches='tight',
facecolor='white', edgecolor='none')
plt.close()

print(f"✅ {name} 纯净黑白版决策树已生成: {save_name}")

print("\n 全部完成！")

```

附录 4

介绍：AHP 权重计算代码

```

"""
AHP 指标权重构建
"""

import numpy as np
import pandas as pd

def ahp_weight(priority_scores, criteria_names=None):
    """AHP 局部权重计算 + 一致性检验"""
    n = len(priority_scores)
    matrix = np.zeros((n, n))
    for i in range(n):
        for j in range(n):
            matrix[i, j] = priority_scores[i] / priority_scores[j]

    eigenvalues, eigenvectors = np.linalg.eig(matrix)
    max_eig_idx = np.argmax(np.real(eigenvalues))
    lambda_max = np.real(eigenvalues[max_eig_idx])
    weight = np.real(eigenvectors[:, max_eig_idx])
    weight = weight / np.sum(weight)

    CI = (lambda_max - n) / (n - 1)
    RI_dict = {1:0, 2:0, 3:0.58, 4:0.90, 5:1.12, 6:1.24, 7:1.32,
8:1.41, 9:1.45}
    RI = RI_dict.get(n, 1.45)
    CR = CI / RI if RI > 0 else 0

    print(f"λ_max = {lambda_max:.4f} | CI = {CI:.4f} | CR = {CR:.4f} {'✅ 通过' if CR < 0.1 else '❌ 需要调整'}")

    if criteria_names:

```

```

        for name, w in zip(criteria_names, weight):
            print(f"        └── {name:14} : {w:.4f}
({w*100:5.2f}%)"")
        return weight

#1. 定义层次结构
hierarchy = {
    "A 健康需求": {
        "weight": 0.5396,
        "sub_indicators": ["年龄", "身体状况"],
        "priorities": [1, 3]
    },
    "B 经济需求": {
        "weight": 0.2970,
        "sub_indicators": ["每月养老金", "房产数目", "是否低保户",
"人才层级"],
        "priorities": [3, 1, 7, 5]
    },
    "C 家庭支持": {
        "weight": 0.1634,
        "sub_indicators": ["子女个数", "配偶情况"],
        "priorities": [1, 3]
    }
}

#2. 读取人员数据
file_path = "D:\math model\数学建模 1.xlsx"

df_data = pd.read_excel(file_path, sheet_name="数学建模")
print(f"✅ 成功读取人员数据, 共 {len(df_data)} 条记录")

# 3. 计算权重
print("\n" + "="*70)
print("AHP 权重计算中...\n")

global_weights = {}

for levell_name, data in hierarchy.items():
    print(f"【{levell_name}】 (一级权重: {data['weight']:.4f})")

    local_w = ahp_weight(data["priorities"],
data["sub_indicators"])
    data["local_weights"] = local_w

```

```

# 计算全局权重
for sub_name, lw in zip(data["sub_indicators"], local_w):
    gw = lw * data["weight"]
    global_weights[sub_name] = gw
    print(f"        → 全局权重: {gw:.5f} ({gw*100:6.2f}%)")
print("-" * 60)

#4. 全局权重排序
print("\n🌀 最终全局权重排序（从高到低）:")
sorted_global = sorted(global_weights.items(), key=lambda x: x[1],
reverse=True)
for name, w in sorted_global:
    print(f"{name:14} : {w:.5f} ({w*100:6.2f}%)")

# 5. 指标列名映射
col_map = {
    "年龄": "年龄",
    "身体状况": "身体状况",
    "每月养老金": "每月养老金（元）",
    "房产数目": "房产数目",
    "是否低保户": "是否为低保户",
    "人才层级": "是否为国家、省、市级人才",    # 注意: Excel 中是这个列
    "子女个数": "子女个数",
    "配偶情况": "当前是否是配偶配偶是否健在"
}

# 6. 输出与保存
print("\n✅ 前 10 名结果:")
print(df_data[["人员序号", "综合得分", "排名"]].head(10).to_string(index=False))

# 保存结果
output_file = "数学建模_最终评分结果.xlsx"
with pd.ExcelWriter(output_file) as writer:
    df_data.to_excel(writer, sheet_name="评分结果", index=False)

    # 保存权重表
    weight_df = pd.DataFrame(sorted_global, columns=["指标名称", "全局权重"])
    weight_df.to_excel(writer, sheet_name="全局权重", index=False)

```

```
print(f"\n 所有结果已保存至: {output_file}")
```

附录 5

介绍：熵权计算代码

```
# -*- coding: utf-8 -*-
"""
熵权
"""

import pandas as pd
import numpy as np

# 1. 读取数据
file_path = "D:\math model\数学建模 1.xlsx" # 修改为你的实际路径

# 读取人员数据
df = pd.read_excel(file_path, sheet_name="数学建模")

# 读取指标对应关系（可选，用于参考）
ahp = pd.read_excel(file_path, sheet_name="AHP 指标构建")
print("指标对应关系: ")
print(ahp)

# 2. 选取你需要的指标
# 根据图片和 AHP 表，你选择的二级指标对应列名如下：
indicators = {
    '年龄': '年龄',
    '身体状况': '身体状况',
    '每月养老金（元）': '每月养老金（元）',
    '房产数目': '房产数目',
    '是否为低保户': '是否为低保户',
    '子女个数': '子女个数',
    '人才层级': '是否为国家、省、市级人才',
    '配偶情况': '当前是否是配偶配偶是否健在'
}

# 提取数据矩阵
data = df[list(indicators.values())].copy()

# 重命名列为中文方便阅读
data.columns = list(indicators.keys())
print("\n 数据预览: ")
print(data.head())
```

```

# 3. 数据预处理
# 处理缺失值（如果有，用 0 或均值填充，根据实际意义决定）
data = data.fillna(0)

# 4. 熵权法核心函数
def entropy_weight(df):
    # 转为 numpy 数组
    X = df.values.astype(float)
    m, n = X.shape # m=样本数, n=指标数

    # 1. 数据标准化（正向化）
    # 对于所有指标，我们统一做“越大越好”的标准化（极差法）
    X_std = np.zeros_like(X)
    for j in range(n):
        col = X[:, j]
        min_val = col.min()
        max_val = col.max()
        if max_val == min_val: # 防止全相同
            X_std[:, j] = 1
        else:
            # 正向指标：越大越好
            X_std[:, j] = (col - min_val) / (max_val - min_val)

    # 2. 计算比例 p_ij
    X_std = X_std + 1e-10 # 避免 log(0)
    p = X_std / X_std.sum(axis=0)

    # 3. 计算熵值 e_j
    k = 1 / np.log(m)
    e = -k * np.sum(p * np.log(p), axis=0)

    # 4. 计算差异系数 d_j
    d = 1 - e

    # 5. 计算权重 w_j
    w = d / d.sum()

    # 结果
    result = pd.DataFrame({
        '指标': df.columns,
        '熵值': e,
        '差异系数': d,

```

```

        '熵权': w
    })
    return result.round(4)

# 5. 计算并输出
weights = entropy_weight(data)
print("\n=== 熵权计算结果 ===")
print(weights)

# 保存结果
weights.to_excel("熵权结果.xlsx", index=False)
print("\n 结果已保存为：熵权结果.xlsx")

```

附录 6

介绍：问题二分配方案

```

# -*- coding: utf-8 -*-
"""
数学建模 - 养老金补贴分配系统
融合 MPI 分类 + AHP + 熵权 + 敏感性分析 + 最大余额法整数规划
"""

import os
import pandas as pd
import numpy as np

# 1. 参数设置
TOTAL_BUDGET = 200000      # 总预算 20 万元
HIGH_DEPRIVED_AMOUNT = 1500 # 高剥夺群体固定补贴标准
LAMBDA_S = [0.5, 0.6, 0.7] # 敏感性分析权重调节参数

# 2. 读取数据
# 使用原始字符串 r'' 防止 Windows 路径中的斜杠被误判为转义字符
data_path = r'D:\math model\数学建模 1.xlsx'

if not os.path.exists(data_path):
    raise FileNotFoundError(f"未找到指定的 Excel 文件，请检查路径是否正确：{data_path}")

df = pd.read_excel(data_path, sheet_name='数学建模')

# 列名简化与规范化
df = df.rename(columns={
    '每月养老金（元）': '养老金',

```

```

    '当前是否是配偶配偶是否健在': '配偶情况',
    '是否为低保户': '低保户',
    '是否为国家、省、市级人才': '人才层级'
})

print(f"数据读取成功! 总样本人数: {len(df)} 人")

# 3. MPI 剥夺分数计算
def compute_deprivation_score(row):
    """
    计算MPI剥夺分数（多维贫困指数思路）
    """
    score = 0
    if row['养老金'] <= 1000:                # 经济困难
        score += 1
    if row['子女个数'] == 0:                # 无子女
        score += 1
    if row['房产数目'] == 0:                # 无房产
        score += 1
    if row['低保户'] == 1:                  # 低保户
        score += 1
    if row['身体状况'] != 1:                # 非健康
        score += 1
    if row['配偶情况'] == 0:                # 配偶不在
        score += 1
    if row['年龄'] >= 80:                    # 高龄
        score += 1
    return score

df['MPI_剥夺分数'] = df.apply(compute_deprivation_score, axis=1)

# 划分高剥夺群体（MPI > 4）与普通群体
high_deprived = df[df['MPI_剥夺分数'] > 4].copy()
remaining = df[df['MPI_剥夺分数'] <= 4].copy()

# 高剥夺群体固定发放金额
high_deprived['补贴'] = HIGH_DEPRIVED_AMOUNT
high_cost = len(high_deprived) * HIGH_DEPRIVED_AMOUNT
remain_budget = TOTAL_BUDGET - high_cost

print("初始筛查完成:")
print(f"    - 特别困难高剥夺老人（MPI > 4）: {len(high_deprived)} 人, 共支出 {high_cost} 元")
print(f"    - 纳入精细化分配的普通老人: {len(remaining)} 人")

```



```

print(f"    - 普通老人可用剩余预算: {remain_budget} 元")

if remain_budget < 0:
    print("警告: 高剥夺群体的固定补贴总额已超过 20 万总预算, 请重新调整预算或补贴标准!")

# 4. 指标与权重设定
indicators = ['年龄', '身体状况', '养老金', '房产数目', '低保户', '人才层级', '子女个数', '配偶情况']
w_ahp = [0.1349, 0.4047, 0.0557, 0.0186, 0.1299, 0.0928, 0.0409, 0.1225]
w_entropy = [0.0302, 0.1081, 0.0342, 0.0306, 0.3507, 0.31, 0.0382, 0.0980]
pos_neg = [1, 1, 0, 0, 1, 1, 0, 0] # 1 代表正向指标, 0 代表负向指标

# 5. 数据归一化
def min_max_normalize(series, positive=True):
    """标准 Min-Max 归一化, 加入极小值防止分母为 0"""
    min_val, max_val = series.min(), series.max()
    if max_val == min_val:
        return np.zeros(len(series))
    if positive:
        return (series - min_val) / (max_val - min_val + 1e-8)
    else:
        return (max_val - series) / (max_val - min_val + 1e-8)

binary_cols = {'低保户', '配偶情况'} # 二值变量保持 0/1 业务含义, 不参与动态极差缩放
norm_data = pd.DataFrame(index=remaining.index)

for i, col in enumerate(indicators):
    if col in binary_cols:
        norm_data[col] = remaining[col].astype(float)
    else:
        norm_data[col] = min_max_normalize(remaining[col], positive=pos_neg[i])

print("数据归一化完成(二值变量已做保留处理)。")

# 6. 最大余额法分配算法
def distribute_budget(scores, budget, multiple=100):
    """
    采用最大余额法 (Largest Remainder Method) 变体:

```

```

1. 确保每个人分配金额是 multiple (100) 的整倍数
2. 确保最终总金额严格等于 budget, 误差为 0
3. 优先补偿因取整损失精度的个体, 兼顾数学公平性
"""

if len(scores) == 0 or budget <= 0:
    return np.zeros(len(scores), dtype=int)

total_score = scores.sum()
if total_score == 0:
    # 如果所有人综合得分均为 0, 则均分预算
    base = (budget // len(scores)) // multiple * multiple
    alloc = np.full(len(scores), base)
    remainder = budget - alloc.sum()
    num_to_adjust = int(remainder // multiple)
    alloc[:num_to_adjust] += multiple
    return alloc

# Step 1: 计算按得分比例划分的精确理论份额
exact_shares = (scores / total_score) * budget

# Step 2: 向下取整到 100 的倍数 (进行基础安全分配, 确保绝不超
支)
alloc = (exact_shares // multiple) * multiple
alloc = alloc.astype(int)

# Step 3: 计算由于取整而被“亏欠”的残差 (余数)
remainders = exact_shares - alloc

# Step 4: 计算当前的预算缺口
current_sum = alloc.sum()
diff = budget - current_sum

# 计算需要补齐的 100 元面额的个数
num_slots_to_fill = int(round(diff / multiple))

# Step 5: 按照残差从大到小排序, 将缺口依次补齐
if num_slots_to_fill > 0:
    sorted_indices = np.argsort(-remainders) # 获取残差降序排
列的索引
    chosen_indices = sorted_indices[:num_slots_to_fill] # 挑
出前 N 个亏欠最多的人
    alloc[chosen_indices] += multiple # 每个人补偿 100 元

return alloc

```

```

#7. 敏感性分析与融合权重计算
for lam in LAMBDAS:
    scores = np.zeros(len(norm_data))
    # 线性组合 AHP 权重与熵权
    w_fused = np.array(w_ahp) * lam + np.array(w_entropy) * (1 - lam)

    # 计算每位普通老人的综合发展得分
    for i, col in enumerate(indicators):
        scores += norm_data[col].values * w_fused[i]

    remaining[f'融合得分_λ {lam}'] = scores

    # 调用最大余额法进行金额整算分配
    alloc = distribute_budget(scores, remain_budget, multiple=100)
    remaining[f'补贴_λ {lam}'] = alloc

    # 为高剥夺群体同步创建对应的敏感性列，保持合并结构一致
    high_deprived[f'融合得分_λ {lam}'] = 1.0 # 高剥夺群体不参与评分，设为常数项
    high_deprived[f'补贴_λ {lam}'] = HIGH_DEPRIVED_AMOUNT

# 8. 数据合并与排序
final_df = pd.concat([high_deprived, remaining], axis=0, sort=False)
final_df = final_df.sort_values(by='人员序号').reset_index(drop=True)

# 9. 最终预算闭环检查与保存
print("\n" + "="*70)
print(" 最终预算执行与敏感性分析检查")
print("="*70)
print(f"总预算总体控制目标: {TOTAL_BUDGET} 元")

for lam in LAMBDAS:
    total_alloc = final_df[f'补贴_λ {lam}'].sum()
    error = total_alloc - TOTAL_BUDGET
    print(f"方案 [ λ = {lam} ] 实际总发放: {total_alloc} 元 | 统计误差: {error} 元")

# 检查是否每个人都是 100 的倍数
for lam in LAMBDAS:
    non_100_count = np.sum(final_df[f'补贴_λ {lam}'] % 100 != 0)

```

```

        if non_100_count == 0:
            print(f"方案 [  $\lambda$  = {lam} ] 约束合规性：所有人补贴金额均满足 100 的倍数要求。")
        else:
            print(f"方案 [  $\lambda$  = {lam} ] 约束合规性：存在 {non_100_count} 条数据不满足 100 的倍数！")

# 保存结果至 Excel
output_path = '养老金补贴分配最终结果.xlsx'
with pd.ExcelWriter(output_path) as writer:
    final_df.to_excel(writer, sheet_name='最终分配结果',
index=False)
    high_deprived.to_excel(writer, sheet_name='高剥夺群体明细',
index=False)
    remaining.to_excel(writer, sheet_name='普通群体精细分配',
index=False)

print("="*70)
print(f"建模数据处理完毕！结果已成功写回：{output_path}")
print("运行前 10 条样本预览：")
show_cols = ['人员序号', 'MPI_剥夺分', '融合得分_ $\lambda$  0.5', '补贴_ $\lambda$  0.5', '补贴_ $\lambda$  0.6', '补贴_ $\lambda$  0.7']
# 兼容列名映射输出
preview_df = final_df.copy()
if 'MPI_剥夺分数' in preview_df.columns:
    preview_df = preview_df.rename(columns={'MPI_剥夺分数': 'MPI_剥夺分'})
print(preview_df[show_cols].head(10).to_string(index=False))

```

附录 7

介绍：问题三分配方案

```

# -*- coding: utf-8 -*-
"""
第三问 - 最终版
"""

import pandas as pd
import numpy as np

# 1. 数据加载与基础参数设定
file_name = 'D:\\math model\\code\\第三问.xlsx' # ← 修改为你实际路径
df = pd.read_excel(file_name)

```

```

O = df['方案二分配金额'].values.astype(float)
I = df['理想金额'].values.astype(float)
N = len(df)

ORIGINAL_TOTAL = 200000.0
TARGET_TOTAL = 300000.0
TARGET_INCREMENT = TARGET_TOTAL - ORIGINAL_TOTAL # 100,000 元

# 2. 计算差额比例 d_i 并筛选前 20% 的正向调整对象

d = np.zeros(N)

for i in range(N):
    if O[i] == 0: # 新增: O_i 为 0 时, d_i 直接取 1
        d[i] = 1.0
    else:
        d[i] = (I[i] - O[i]) / O[i]

# 将计算好的偏差数组 d 显式写入 DataFrame, 命名为新列 '差额比例_di'
df['差额比例_di'] = d

# 筛选策略: 只拉取差额比例为正(理想状态下拿少了)的人, 并降序排列
df_positive = df[df['差额比例_di'] > 0].copy()
df_positive = df_positive.sort_values(by='差额比例_di',
ascending=False)

max_adjust_count = int(0.2 * N) # 51 人
adjusted_indices = df_positive.iloc[:max_adjust_count]['人员序号']
                        .values - 1

# 3. 初始启发式金额计算 + 信赖保护原则截断

A_raw = 0.copy()

for i in adjusted_indices:
    if O[i] > 0:
        lower = 0.75 * O[i]
        upper = 2.25 * O[i]
        A_raw[i] = min(max(I[i], lower), upper)
    else:
        A_raw[i] = min(I[i], 500.0)

theoretical_increment = A_raw - O
total_theoretical_inc = theoretical_increment.sum()

```

```

print(f"【步骤 2 成功】51 人初始理论增额总和为：
{total_theoretical_inc:.2f} 元。")

# 4. 改进型最大余额法 (Largest Remainder Method)
weights = np.zeros(N)
for i in adjusted_indices:
    if total_theoretical_inc > 0:
        weights[i] = theoretical_increment[i] /
total_theoretical_inc

total_units_to_distribute = int(TARGET_INCREMENT / 100)    # 1000

exact_units = weights * total_units_to_distribute
floor_units = np.floor(exact_units).astype(int)
remainders = exact_units - floor_units

allocated_units_so_far = np.sum(floor_units)
units_left = total_units_to_distribute - allocated_units_so_far
print(f"【步骤 3 推进】第一轮基础分配消耗
{allocated_units_so_far*100} 元，剩余 {units_left} 个 '100 元' 单
位。")

# 第二轮余数分配（带边界保护）
remainder_list = [(i, remainders[i]) for i in adjusted_indices]
remainder_list.sort(key=lambda x: x[1], reverse=True)

final_units = floor_units.copy()
units_allocated_in_round2 = 0

for idx, rem in remainder_list:
    if units_allocated_in_round2 >= units_left:
        break

    test_amount = 0[idx] + (final_units[idx] + 1) * 100

    is_safe = False
    if 0[idx] > 0:
        if 0.75 * 0[idx] <= test_amount <= 2.25 * 0[idx]:
            is_safe = True
    else:
        if test_amount <= 500.0:
            is_safe = True

```

```

        if is_safe:
            final_units[idx] += 1
            units_allocated_in_round2 += 1

print(f"【步骤 4 成功】第二轮通过安全校验，分发出
{units_allocated_in_round2 * 100} 元。")

# 5. 结果生成 + 输出 Excel
df['最终分配金额'] = 0 + (final_units * 100)
df['金额变化量'] = df['最终分配金额'] - 0
df['是否调整'] = (df['金额变化量'] != 0).astype(int)

#输出 Excel
output_file = '第三问_最终分配方案.xlsx'

df.to_excel(output_file, index=False)

print(f"\n 结果已成功输出至: {output_file}")
print(f"总分配金额: {df['最终分配金额'].sum():.2f} 元 (目标 300000
元)")
print(f"调整人数: {df['是否调整'].sum()} 人")

```

附录 8

介绍：问题四分配方案

```

# -*- coding: utf-8 -*-
"""
数学建模 - 养老金补贴分配系统
融合 MPI 分类 + AHP + 熵权 + 敏感性分析 + 最大余额法整数规划
"""

import os
import pandas as pd
import numpy as np

# 1. 参数设置
TOTAL_BUDGET = 300000      # 总预算 20 万元
HIGH_DEPRIVED_AMOUNT = 1500 # 高剥夺群体固定补贴标准
LAMBDA_S = [0.5, 0.6, 0.7] # 敏感性分析权重调节参数

# 2. 读取数据
# 使用原始字符串 r'' 防止 Windows 路径中的斜杠被误判为转义字符
data_path = r'D:\math model\数学建模 1.xlsx'

```

```

if not os.path.exists(data_path):
    raise FileNotFoundError(f"未找到指定的 Excel 文件，请检查路径是否正确：{data_path}")

df = pd.read_excel(data_path, sheet_name='数学建模')

# 列名简化与规范化
df = df.rename(columns={
    '每月养老金（元）': '养老金',
    '当前是否是配偶配偶是否健在': '配偶情况',
    '是否为低保户': '低保户',
    '是否为国家、省、市级人才': '人才层级'
})

print(f"数据读取成功！总样本人数：{len(df)} 人")

# 3. MPI 剥夺分数计算
def compute_deprivation_score(row):
    """
    计算 MPI 剥夺分数（多维贫困指数思路）
    """
    score = 0
    if row['养老金'] <= 1000:                # 经济困难
        score += 1
    if row['子女个数'] == 0:                # 无子女
        score += 1
    if row['房产数目'] == 0:                # 无房产
        score += 1
    if row['低保户'] == 1:                  # 低保户
        score += 1
    if row['身体状况'] != 1:                # 非健康
        score += 1
    if row['配偶情况'] == 0:                # 配偶不在
        score += 1
    if row['年龄'] >= 80:                    # 高龄
        score += 1
    return score

df['MPI_剥夺分数'] = df.apply(compute_deprivation_score, axis=1)

# 划分高剥夺群体（MPI > 4）与普通群体
high_deprived = df[df['MPI_剥夺分数'] > 4].copy()
remaining = df[df['MPI_剥夺分数'] <= 4].copy()

```



```

# 高剥夺群体固定发放金额
high_deprived['补贴'] = HIGH_DEPRIVED_AMOUNT
high_cost = len(high_deprived) * HIGH_DEPRIVED_AMOUNT
remain_budget = TOTAL_BUDGET - high_cost

print("初始筛查完成:")
print(f"    - 特别困难高剥夺老人 (MPI > 4): {len(high_deprived)} 人, 共支出 {high_cost} 元")
print(f"    - 纳入精细化分配的普通老人: {len(remaining)} 人")
print(f"    - 普通老人可用剩余预算: {remain_budget} 元")

if remain_budget < 0:
    print("警告: 高剥夺群体的固定补贴总额已超过 20 万总预算, 请重新调整预算或补贴标准!")

# 4. 指标与权重设定
indicators = ['年龄', '身体状况', '养老金', '房产数目', '低保户', '人才层级', '子女个数', '配偶情况']
w_ahp = [0.1349, 0.4047, 0.0557, 0.0186, 0.1299, 0.0928, 0.0409, 0.1225]
w_entropy = [0.0302, 0.1081, 0.0342, 0.0306, 0.3507, 0.31, 0.0382, 0.0980]
pos_neg = [1, 1, 0, 0, 1, 1, 0, 0] # 1 代表正向指标, 0 代表负向指标

# 5. 数据归一化
def min_max_normalize(series, positive=True):
    """标准 Min-Max 归一化, 加入极小值防止分母为 0"""
    min_val, max_val = series.min(), series.max()
    if max_val == min_val:
        return np.zeros(len(series))
    if positive:
        return (series - min_val) / (max_val - min_val + 1e-8)
    else:
        return (max_val - series) / (max_val - min_val + 1e-8)

binary_cols = {'低保户', '配偶情况'} # 二值变量保持 0/1 业务含义, 不参与动态极差缩放
norm_data = pd.DataFrame(index=remaining.index)

for i, col in enumerate(indicators):
    if col in binary_cols:
        norm_data[col] = remaining[col].astype(float)

```

```

else:
    norm_data[col] = min_max_normalize(remaining[col],
positive=pos_neg[i])

print("数据归一化完成（二值变量已做保留处理）。")

# 6. 最大余额法分配算法
def distribute_budget(scores, budget, multiple=100):
    """
    采用最大余额法（Largest Remainder Method）变体：
    1. 确保每个人分配金额是 multiple（100）的整倍数
    2. 确保最终总金额严格等于 budget，误差为 0
    3. 优先补偿因取整损失精度的个体，兼顾数学公平性
    """

    if len(scores) == 0 or budget <= 0:
        return np.zeros(len(scores), dtype=int)

    total_score = scores.sum()
    if total_score == 0:
        # 如果所有人综合得分均为 0，则均分预算
        base = (budget // len(scores)) // multiple * multiple
        alloc = np.full(len(scores), base)
        remainder = budget - alloc.sum()
        num_to_adjust = int(remainder // multiple)
        alloc[:num_to_adjust] += multiple
        return alloc

    # Step 1: 计算按得分比例划分的精确理论份额
    exact_shares = (scores / total_score) * budget

    # Step 2: 向下取整到 100 的倍数（进行基础安全分配，确保绝不超
    支）
    alloc = (exact_shares // multiple) * multiple
    alloc = alloc.astype(int)

    # Step 3: 计算由于取整而被“亏欠”的残差（余数）
    remainders = exact_shares - alloc

    # Step 4: 计算当前的预算缺口
    current_sum = alloc.sum()
    diff = budget - current_sum

    # 计算需要补齐的 100 元面额的个数
    num_slots_to_fill = int(round(diff / multiple))

```

```

# Step 5: 按照残差从大到小排序, 将缺口依次补齐
if num_slots_to_fill > 0:
    sorted_indices = np.argsort(-remainders) # 获取残差降序排列的索引
    chosen_indices = sorted_indices[:num_slots_to_fill] # 挑出前 N 个亏欠最多的人
    alloc[chosen_indices] += multiple # 每个人补偿 100 元

return alloc

#7. 敏感性分析与融合权重计算
for lam in LAMBDAS:
    scores = np.zeros(len(norm_data))
    # 线性组合 AHP 权重与熵权
    w_fused = np.array(w_ahp) * lam + np.array(w_entropy) * (1 - lam)

    # 计算每位普通老人的综合发展得分
    for i, col in enumerate(indicators):
        scores += norm_data[col].values * w_fused[i]

    remaining[f'融合得分_λ {lam}'] = scores

    # 调用最大余额法进行金额整算分配
    alloc = distribute_budget(scores, remain_budget, multiple=100)
    remaining[f'补贴_λ {lam}'] = alloc

    # 为高剥夺群体同步创建对应的敏感性列, 保持合并结构一致
    high_deprived[f'融合得分_λ {lam}'] = 1.0 # 高剥夺群体不参与评分, 设为常数项
    high_deprived[f'补贴_λ {lam}'] = HIGH_DEPRIVED_AMOUNT

# 8. 数据合并与排序
final_df = pd.concat([high_deprived, remaining], axis=0, sort=False)
final_df = final_df.sort_values(by='人员序号').reset_index(drop=True)

# 9. 最终预算闭环检查与保存
print("\n" + "="*70)
print("最终预算执行与敏感性分析检查")
print("="*70)
print(f"总预算总体控制目标: {TOTAL_BUDGET} 元")

```

```

for lam in LAMBDA:
    total_alloc = final_df[f' 补贴_λ {lam}'].sum()
    error = total_alloc - TOTAL_BUDGET
    print(f"方案 [ λ = {lam} ] 实际总发放: {total_alloc} 元 | 统计误差: {error} 元")

# 检查是否每个人都是 100 的倍数
for lam in LAMBDA:
    non_100_count = np.sum(final_df[f' 补贴_λ {lam}'] % 100 != 0)
    if non_100_count == 0:
        print(f"方案 [ λ = {lam} ] 约束合规性: 所有人补贴金额均满足 100 的倍数要求。")
    else:
        print(f"方案 [ λ = {lam} ] 约束合规性: 存在 {non_100_count} 条数据不满足 100 的倍数!")

# 保存结果至 Excel
output_path = '4. 养老金补贴分配最终结果.xlsx'
with pd.ExcelWriter(output_path) as writer:
    final_df.to_excel(writer, sheet_name='最终分配结果',
index=False)
    high_deprived.to_excel(writer, sheet_name='高剥夺群体明细',
index=False)
    remaining.to_excel(writer, sheet_name='普通群体精细分配',
index=False)

print("="*70)
print(f"建模数据处理完毕! 结果已成功写回: {output_path}")
print("运行前 10 条样本预览:")
show_cols = ['人员序号', 'MPI_剥夺分', '融合得分_λ 0.5', '补贴_λ 0.5', '补贴_λ 0.6', '补贴_λ 0.7']
# 兼容列名映射输出
preview_df = final_df.copy()
if 'MPI_剥夺分数' in preview_df.columns:
    preview_df = preview_df.rename(columns={'MPI_剥夺分数': 'MPI_剥夺分'})
print(preview_df[show_cols].head(10).to_string(index=False))

```